

PROJEKT ZALICZENIOWY · BAZY DANYCH (PL/SQL)

# TechSklep

Sklep internetowy — dokumentacja techniczna i materiały do obrony

Oracle Database 19c · PL/SQL · Python / Flask · Bootstrap 5

Autorzy: [Imie Nazwisko], [Imie Nazwisko] · 2026

# Spis treści

---

|                                                             |    |
|-------------------------------------------------------------|----|
| <b>1. Wprowadzenie</b>                                      | 4  |
| 1.1. Cel i wymagania                                        | 4  |
| 1.2. Jak czytać ten dokument                                | 4  |
| <b>2. Architektura systemu</b>                              | 5  |
| <b>3. Stos technologiczny</b>                               | 6  |
| <b>4. Model danych (ERD)</b>                                | 7  |
| <b>5. Schemat bazy - projekt.sql (omówienie pełne)</b>      | 8  |
| 5.1. Blok czyszczący (DROP)                                 | 8  |
| 5.2. Sekwencje                                              | 8  |
| 5.3. Tabela KLIENCI                                         | 9  |
| 5.4. Tabela KATEGORIE                                       | 10 |
| 5.5. Tabela PRODUKTY                                        | 10 |
| 5.6. Tabela ZAMOWIENIA                                      | 11 |
| 5.7. Tabela POZYCJE_ZAMOWIENIA                              | 11 |
| 5.8. Tabela PLATNOSCI                                       | 12 |
| 5.9. Tabela RECENZJE                                        | 13 |
| 5.10. Tabela LOG_OPERACJI                                   | 13 |
| 5.11. Indeksy                                               | 14 |
| 5.12. Triggery nadające identyfikatory                      | 14 |
| <b>6. Dane testowe - 02_dane_testowe.sql</b>                | 15 |
| 6.1. Czyszczenie danych                                     | 15 |
| 6.2. Wstawianie danych                                      | 15 |
| 6.3. Dociąganie sekwencji                                   | 15 |
| <b>7. Pakiety PL/SQL - 03_pakiety.sql (omówienie pełne)</b> | 17 |
| 7.1. Specyfikacja pakietu P_ZAMOWIENIA                      | 17 |
| 7.2. Ciało pakietu – funkcja zloz_zamowienie                | 18 |
| 7.3. Procedura dodaj_pozycje                                | 19 |
| 7.4. Funkcje oblicz_sume_zamowienia i przelicz_sume         | 20 |
| 7.5. Procedura zmien_status (maszyna stanów)                | 20 |
| 7.6. anuluj_zamowienie i dodaj_platnosc                     | 21 |
| 7.7. Pakiet P_KLIENCI – rejestracja                         | 22 |
| 7.8. Obsługa błędów w pakietach                             | 22 |
| <b>8. Triggery - 04_triggery.sql (omówienie pełne)</b>      | 24 |
| 8.1. Kontrola stanu przy dodawaniu pozycji                  | 24 |
| 8.2. Zdejmowanie i zwrot stanu magazynowego                 | 24 |

|                                                                        |           |
|------------------------------------------------------------------------|-----------|
| 8.3. Triggery audytowe . . . . .                                       | 25        |
| <b>9. Role i uprawnienia - 05_role.sql (omowienie pelne)</b> . . . . . | <b>27</b> |
| 9.1. Widoki . . . . .                                                  | 27        |
| 9.2. Sprzatanie i tworzenie rol . . . . .                              | 27        |
| 9.3. Nadawanie uprawnień . . . . .                                     | 28        |
| 9.4. Uzytkownicy bazy . . . . .                                        | 29        |
| <b>10. Testy - 90_testy.sql (omowienie pelne)</b> . . . . .            | <b>30</b> |
| 10.1. Test 1 i 2 - magazyn po oplaceniu i anulowaniu . . . . .         | 30        |
| 10.2. Test 3 - przekroczenie stanu . . . . .                           | 31        |
| 10.3. Test 4 - niedozwolona zmiana statusu . . . . .                   | 31        |
| 10.4. Test 5 - trigger audytowy . . . . .                              | 31        |
| <b>11. Aplikacja kliencka (Flask)</b> . . . . .                        | <b>33</b> |
| 11.1. Konfiguracja - config.py . . . . .                               | 33        |
| 11.2. Warstwa dostepu - db.py (pomocnicze) . . . . .                   | 33        |
| 11.3. Logowanie i bcrypt - app.py . . . . .                            | 34        |
| 11.4. Szablony i wyglad . . . . .                                      | 35        |
| <b>12. Bezpieczenstwo</b> . . . . .                                    | <b>36</b> |
| <b>13. Zgodnosc z wymaganiami</b> . . . . .                            | <b>37</b> |
| <b>14. Decyzje projektowe do obrony</b> . . . . .                      | <b>38</b> |
| <b>15. Mozliwe pytania na obronie</b> . . . . .                        | <b>39</b> |
| <b>16. Instrukcja uruchomienia</b> . . . . .                           | <b>40</b> |
| 16.1. Baza (SQL Developer) . . . . .                                   | 40        |
| 16.2. Aplikacja . . . . .                                              | 40        |
| <b>17. Prezentacja</b> . . . . .                                       | <b>41</b> |

# 1. Wprowadzenie

---

Dokument opisuje projekt **TechSklep** – sklep internetowy zrealizowany jako baza danych Oracle 19c z logiką w PL/SQL oraz aplikacja kliencka we Flasku. Część bazodanowa (rozdziały 4–10) została opisana wyczerpująco, kod jest omawiany linia po linii, bo to on jest podstawą obrony.

## 1.1. Cel i wymagania

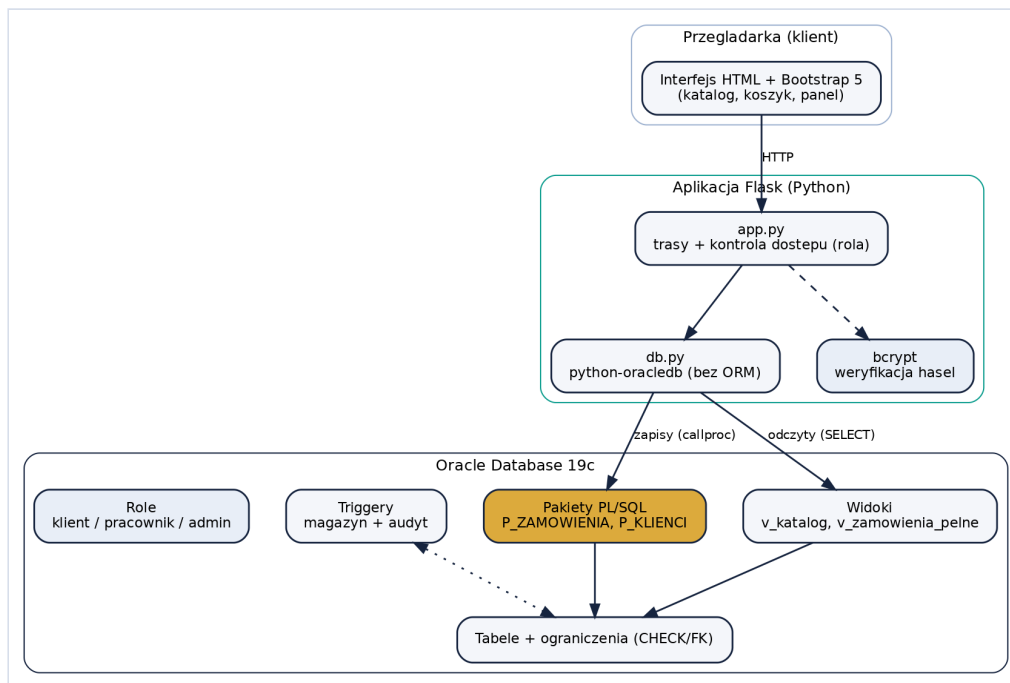
- schemat bazy danych oraz aplikacja bazodanowa,
- logika w PL/SQL: pakiety i triggerzy,
- różne role użytkowników (klient, pracownik, admin) z różnymi uprawnieniami,
- aplikacja kliencka komunikująca się z bazą, hasła hashowane (bcrypt), bez ORM,
- prezentacja oraz procentowy wkład członków grupy.

## 1.2. Jak czytać ten dokument

Przy każdym fragmencie kodu znajduje się listing (z numerami linii) oraz omówienie. Ważniejsze pojęcia PL/SQL są wyjaśnione w ramach “Pojęcie”. Numery linii w opisach odpowiadają numerom w listingach.

## 2. Architektura systemu

System ma trzy warstwy: przeglądarka (Bootstrap), aplikacja Flask oraz baza Oracle. Aplikacja jest cienka – cała logika biznesowa znajduje się w bazie.



Rysunek 2.1. Architektura trojwarstwowa.

Zasada: **zapisy idą przez pakiety PL/SQL** (callproc/callfunc), a odczyty przez zapytania SELECT (zwykle z widoków).

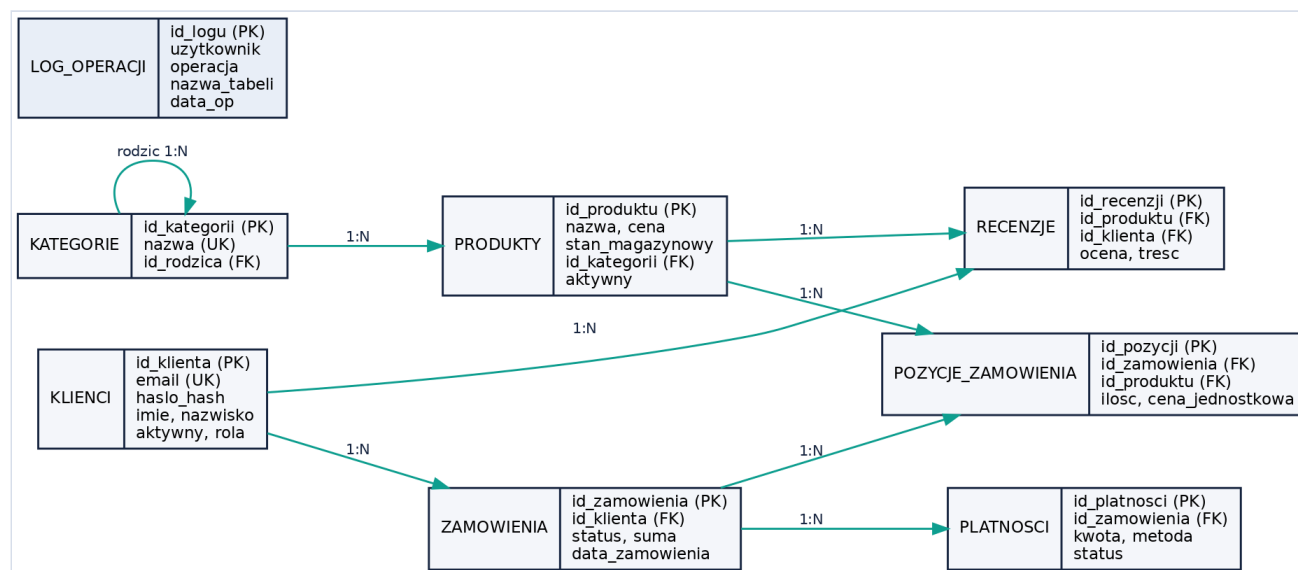
### 3. Stos technologiczny

---

| Warstwa        | Technologia            | Rola                          |
|----------------|------------------------|-------------------------------|
| Baza           | Oracle Database 19c    | dane + logika PL/SQL          |
| Narzedzie      | SQL Developer          | uruchamianie skryptow         |
| Backend        | Python 3 + Flask       | serwer aplikacji              |
| Sterownik      | python-oracledb (thin) | polaczenie bez Instant Client |
| Frontend       | Bootstrap 5 (CDN)      | wyglad                        |
| Bezpieczenstwo | bcrypt                 | hashowanie hasel              |

## 4. Model danych (ERD)

Baza składa się z osmiu tabel. Diagram pokazuje encje i relacje jeden-do-wielu (1:N).



Rysunek 4.1. Diagram związków encji (ERD).

Klient składa wiele zamówień i pisze recenzje; kategoria może mieć podkategorie i wiele produktów; zamówienie składa się z pozycji i ma płatności; produkt występuje w pozycjach i ma recenzje. LOG\_OPERACJI nie ma kluczy obcych – wypełniają go triggery audytowe. Szczegółowy opis każdej kolumny i ograniczenia znajduje się w rozdziale 5.

## 5. Schemat bazy - projekt.sql (omowienie pelne)

Skrypt tworzy caly schemat: usuwa stare obiekty, tworzy sekwencje, osiem tabel z ograniczeniami, indeksy oraz triggery nadajace identyfikatory. Jest idempotentny – mozna go uruchamiac wielokrotnie.

### 5.1. Blok czyszczacy (DROP)

```
4 -- sprzatanie zeby mozna bylo odpalic skrypt drugi raz bez bledow
5 BEGIN
6   FOR t IN (SELECT table_name FROM user_tables WHERE table_name IN
7             ('LOG_OPERACJI', 'RECENZJE', 'PLATNOSCI', 'POZYCJE_ZAMOWIENIA',
8             'ZAMOWIENIA', 'PRODUKTY', 'KATEGORIE', 'KLIENCI'))
9   LOOP
10    EXECUTE IMMEDIATE 'DROP TABLE ' || t.table_name || ' CASCADE CONSTRAINTS';
11  END LOOP;
12
13  FOR s IN (SELECT sequence_name FROM user_sequences WHERE sequence_name IN
14            ('SEQ_KLIENCI', 'SEQ_KATEGORIE', 'SEQ_PRODUKTY', 'SEQ_ZAMOWIENIA',
15            'SEQ_POZYCJE', 'SEQ_PLATNOSCI', 'SEQ_RECENZJE', 'SEQ_LOG'))
16  LOOP
17    EXECUTE IMMEDIATE 'DROP SEQUENCE ' || s.sequence_name;
18  END LOOP;
19 END;
20 /
```

- **Linia 5 (BEGIN)** – początek anonimowego bloku PL/SQL (kod bez nazwy, wykonywany od razu).
- **Linie 6-8** – petla `FOR t IN (SELECT ...)` przechodzi po widoku słownikowym `user_tables` i wybiera tabele projektu (warunek `IN (...)`).
- **Linia 10** – `EXECUTE IMMEDIATE 'DROP TABLE ' || t.table_name || ' CASCADE CONSTRAINTS'` wykonuje dynamicznie polecenie DDL; `CASCADE CONSTRAINTS` usuwa też powiazane klucze obce, dzięki czemu kolejnosc usuwania tabel nie ma znaczenia.
- **Linie 13-18** – analogiczna petla usuwa sekwencje (słownik `user_sequences`).
- **Linia 19 (END;) i 20 (/)** – koniec bloku; znak `/` w SQL Developer wykonuje blok.

#### Pojecie: blok anonimowy i EXECUTE IMMEDIATE

Blok anonimowy to `BEGIN ... END;` bez nazwy – uruchamiany jednorazowo. `EXECUTE IMMEDIATE` pozwala wykonac polecenie zbudowane z tekstu (tzw. SQL dynamiczny); uzywamy go, bo nie da sie wprost napisac `DROP` w petli.

### 5.2. Sekwencje

```
23 -- sekwencje do generowania ID
24 CREATE SEQUENCE seq_klienci   START WITH 1 INCREMENT BY 1 NOCACHE;
25 CREATE SEQUENCE seq_kategorie START WITH 1 INCREMENT BY 1 NOCACHE;
26 CREATE SEQUENCE seq_produkty  START WITH 1 INCREMENT BY 1 NOCACHE;
27 CREATE SEQUENCE seq_zamowienia START WITH 1 INCREMENT BY 1 NOCACHE;
28 CREATE SEQUENCE seq_pozycje   START WITH 1 INCREMENT BY 1 NOCACHE;
```



```

29 CREATE SEQUENCE seq_platnosci START WITH 1 INCREMENT BY 1 NOCACHE;
30 CREATE SEQUENCE seq_recenzje START WITH 1 INCREMENT BY 1 NOCACHE;
31 CREATE SEQUENCE seq_log START WITH 1 INCREMENT BY 1 NOCACHE;

```

Osiem sekwencji generuje unikalne identyfikatory dla tabel. Każda ma te same parametry:

- `START WITH 1` – pierwsza wygenerowana wartość to 1,
- `INCREMENT BY 1` – kolejne wartości rosną o 1,
- `NOCACHE` – baza nie buforuje wartości z góry; numeracja jest ciągła (bez “dziur” po restarcie), kosztem minimalnej wydajności – w projekcie dydaktycznym to zaleta.

### Pojęcie: sekwencja, NEXTVAL i CURRVAL

Sekwencja to generator liczb. `seq.NEXTVAL` zwraca kolejną wartość (i ją “zużywa”), `seq.CURRVAL` – ostatnio pobrana w tej sesji. W projekcie ID nadawane są przez trigger `BEFORE INSERT` (sekcja 5.12), a nie kolumny `IDENTITY` – to wzorec uczony na wykładzie.

## 5.3. Tabela KLIENCI

```

34 -- klienci sklepu
35 CREATE TABLE klienci (
36   id_klienta      NUMBER(10)      NOT NULL,
37   email           VARCHAR2(100)   NOT NULL,
38   haslo_hash      VARCHAR2(255)   NOT NULL, -- nie trzymamy hasła na czysto
39   imie            VARCHAR2(50)    NOT NULL,
40   nazwisko        VARCHAR2(50)    NOT NULL,
41   telefon         VARCHAR2(20),
42   adres_ulica     VARCHAR2(150),
43   adres_miasto    VARCHAR2(80),
44   adres_kod       VARCHAR2(10),
45   data_rejestracji DATE           DEFAULT SYSDATE NOT NULL,
46   aktywny         CHAR(1)         DEFAULT 'T' NOT NULL, -- T/N zamiast kasowania
47   rola            VARCHAR2(20)    DEFAULT 'KLIENT' NOT NULL, -- KLIENT/PRACOWNIK/ADMIN
48
49   CONSTRAINT pk_klienci PRIMARY KEY (id_klienta),
50   CONSTRAINT uk_klienci_email UNIQUE (email),
51   CONSTRAINT ck_klienci_aktywny CHECK (aktywny IN ('T', 'N')),
52   CONSTRAINT ck_klienci_rola CHECK (rola IN ('KLIENT', 'PRACOWNIK', 'ADMIN')),
53   CONSTRAINT ck_klienci_email CHECK (email LIKE '%@%._%') -- prosta walidacja maila
54 );

```

### Kolumny

- `id_klienta` `NUMBER(10)` `NOT NULL` – klucz główny; wartość nadaje trigger z sekwencji.
- `email` `VARCHAR2(100)` `NOT NULL` – adres e-mail, służy jako login.
- `haslo_hash` `VARCHAR2(255)` `NOT NULL` – hash bcrypt (60 znaków), nigdy hasło jawne; 255 znaków daje zapas na inne algorytmy.
- `imie`, `nazwisko` `VARCHAR2(50)` – dane osobowe, wymagane.
- `telefon` `VARCHAR2(20)`, `adres_ulica/miasto/kod` – dane opcjonalne (mogą być `NULL`).
- `data_rejestracji` `DATE` `DEFAULT SYSDATE` `NOT NULL` – data założenia konta; `DEFAULT SYSDATE` wstawia bieżącą datę, gdy nie podano.
- `aktywny` `CHAR(1)` `DEFAULT 'T'` `NOT NULL` – flaga T/N; konta się nie usuwa, tylko dezaktywuje.
- `rola` `VARCHAR2(20)` `DEFAULT 'KLIENT'` `NOT NULL` – `KLIENT/PRACOWNIK/ADMIN`; jedna tabela obsługuje logowanie klientów i personelu.

## Ograniczenia

- `pk_klienci PRIMARY KEY (id_klienta)` – klucz główny: unikalny i NOT NULL.
- `uk_klienci_email UNIQUE (email)` – nie może być dwóch kont na ten sam e-mail.
- `ck_klienci_aktywny CHECK (aktywny IN ('T','N'))` – dopuszcza tylko T lub N.
- `ck_klienci_rola CHECK (rola IN ('KLIENT','PRACOWNIK','ADMIN'))` – ogranicza role.
- `ck_klienci_email CHECK (email LIKE '%_@%._%')` – prosta walidacja formatu e-mail: co najmniej jeden znak, “@”, znak, kropka, znak (np. a@b.pl).

### Pojecie: nazewnictwo ograniczeń

Wszystkie ograniczenia mają nazwy z prefiksem: `pk_` (klucz główny), `uk_` (unikalność), `fk_` (klucz obcy), `ck_` (CHECK). Dzięki temu komunikat o błędzie wskazuje, które ograniczenie zostało naruszone – łatwiej diagnozować problemy.

## 5.4. Tabela KATEGORIE

```
57 -- kategorie produktow, moga miec rodzica (drzewo)
58 CREATE TABLE kategorie (
59   id_kategorii  NUMBER(10)    NOT NULL,
60   nazwa        VARCHAR2(80)  NOT NULL,
61   opis         VARCHAR2(500),
62   id_rodzica   NUMBER(10),    -- NULL = kategoria glowna
63
64   CONSTRAINT pk_kategorie      PRIMARY KEY (id_kategorii),
65   CONSTRAINT uk_kategorie_nazwa UNIQUE (nazwa),
66   CONSTRAINT fk_kategorie_rodzic FOREIGN KEY (id_rodzica)
67     REFERENCES kategorie (id_kategorii)
```

- `id_kategorii` – PK.
- `nazwa VARCHAR2(80)` + `uk_kategorie_nazwa UNIQUE` – unikalna nazwa kategorii.
- `opis VARCHAR2(500)` – opcjonalny opis.
- `id_rodzica NUMBER(10)` – klucz obcy do tej samej tabeli; NULL oznacza kategorię główną.
- `fk_kategorie_rodzic FOREIGN KEY (id_rodzica) REFERENCES kategorie (id_kategorii)` – relacja do samej siebie, tworząca drzewo kategorii.

### Pojecie: klucz obcy odwolujący się do tej samej tabeli (drzewo)

Kolumna `id_rodzica` wskazuje na `id_kategorii` w tej samej tabeli. Pozwala to budować hierarchie (Elektronika → Laptopy). Kategoria główna ma `id_rodzica = NULL`.

## 5.5. Tabela PRODUKTY

```
71 -- produkty w sklepie
72 CREATE TABLE produkty (
73   id_produktu  NUMBER(10)    NOT NULL,
74   nazwa        VARCHAR2(150) NOT NULL,
75   opis         VARCHAR2(2000),
76   cena         NUMBER(10,2)   NOT NULL,
77   stan_magazynowy NUMBER(10)  DEFAULT 0 NOT NULL,
78   id_kategorii NUMBER(10)    NOT NULL,
79   aktywny      CHAR(1)       DEFAULT 'T' NOT NULL, -- N = ukryty w sklepie
80   data_dodania DATE          DEFAULT SYSDATE NOT NULL,
```

```

81
82 CONSTRAINT pk_produkty PRIMARY KEY (id_produkty),
83 CONSTRAINT fk_produkty_kategoria FOREIGN KEY (id_kategorii)
84 REFERENCES kategorie (id_kategorii),
85 CONSTRAINT ck_produkty_cena CHECK (cena >= 0),
86 CONSTRAINT ck_produkty_stan CHECK (stan_magazynowy >= 0),
87 CONSTRAINT ck_produkty_aktywny CHECK (aktywny IN ('T','N'))
88 );

```

- id\_produkty - PK; nazwa VARCHAR2(150); opis VARCHAR2(2000).
- cena NUMBER(10,2) - cena z dwoma miejscami po przecinku.
- stan\_magazynowy NUMBER(10) DEFAULT 0 - ilość na stanie.
- id\_kategorii + fk\_produkty\_kategoria - klucz obcy do KATEGORIE.
- aktywny CHAR(1) DEFAULT 'T' - N ukrywa produkt w katalogu (widok v\_katalog).
- ck\_produkty\_cena CHECK (cena >= 0) - cena nieujemna.
- ck\_produkty\_stan CHECK (stan\_magazynowy >= 0) - stan nigdy ujemny (kluczowe dla magazynu).

## 5.6. Tabela ZAMOWIENIA

```

91 -- zamówienia klientów
92 CREATE TABLE zamowienia (
93   id_zamowienia NUMBER(10) NOT NULL,
94   id_klienta NUMBER(10) NOT NULL,
95   data_zamowienia DATE DEFAULT SYSDATE NOT NULL,
96   status VARCHAR2(20) DEFAULT 'NOWE' NOT NULL,
97   suma NUMBER(12,2) DEFAULT 0 NOT NULL, -- liczona triggerem/procedura
98   adres_dostawy VARCHAR2(300),
99
100  CONSTRAINT pk_zamowienia PRIMARY KEY (id_zamowienia),
101  CONSTRAINT fk_zamowienia_klient FOREIGN KEY (id_klienta)
102  REFERENCES klienci (id_klienta),
103  -- możliwe statusy: NOWE -> OPLACONE -> WYSLANE -> DOSTARCZONE, albo ANULOWANE
104  CONSTRAINT ck_zamowienia_status CHECK (status IN
105   ('NOWE', 'OPLACONE', 'WYSLANE', 'DOSTARCZONE', 'ANULOWANE')),
106  CONSTRAINT ck_zamowienia_suma CHECK (suma >= 0)
107 );

```

- id\_zamowienia - PK; id\_klienta + fk\_zamowienia\_klient - FK do KLIENCI.
- data\_zamowienia DATE DEFAULT SYSDATE - data złożenia.
- status VARCHAR2(20) DEFAULT 'NOWE' - etap realizacji.
- suma NUMBER(12,2) DEFAULT 0 - wartość zamówienia (liczona przez pakiet).
- adres\_dostawy VARCHAR2(300) - adres wysyłki.
- ck\_zamowienia\_status CHECK (status IN ('NOWE', 'OPLACONE', 'WYSLANE', 'DOSTARCZONE', 'ANULOWANE')) - dopuszcza tylko pięć statusów.
- ck\_zamowienia\_suma CHECK (suma >= 0) - suma nieujemna.

## 5.7. Tabela POZYCJE\_ZAMOWIENIA

```

110 -- pozycje zamówienia (czyli koszyk)
111 -- cene kopiujemy z produktu żeby zmiana ceny później nie zepsuła historii
112 CREATE TABLE pozycje_zamowienia (
113   id_pozycji NUMBER(10) NOT NULL,

```

```

114 id_zamowienia    NUMBER(10)      NOT NULL,
115 id_produktu      NUMBER(10)      NOT NULL,
116 ilosc            NUMBER(6)       NOT NULL,
117 cena_jednostkowa NUMBER(10,2)    NOT NULL,
118
119 CONSTRAINT pk_pozycje PRIMARY KEY (id_pozycji),
120 CONSTRAINT fk_pozycje_zamowienie FOREIGN KEY (id_zamowienia)
121 REFERENCES zamowienia (id_zamowienia) ON DELETE CASCADE,
122 CONSTRAINT fk_pozycje_produkt FOREIGN KEY (id_produktu)
123 REFERENCES produkty (id_produktu),
124 CONSTRAINT ck_pozycje_ilosc CHECK (ilosc > 0),
125 CONSTRAINT ck_pozycje_cena CHECK (cena_jednostkowa >= 0),
126 -- ten sam produkt nie moze byc dwa razy w tym samym zamowieniu
127 CONSTRAINT uk_pozycje_unik UNIQUE (id_zamowienia, id_produktu)
128 );

```

- `id_pozycji` – PK.
- `id_zamowienia` + `fk_pozycje_zamowienie` ... ON DELETE CASCADE – FK do ZAMOWIENIA; usuniecie zamowienia kasuje jego pozycje.
- `id_produktu` + `fk_pozycje_produkt` – FK do PRODUKTU.
- `ilosc` NUMBER(6) + `ck_pozycje_ilosc` CHECK (`ilosc > 0`) – ilosc dodatnia.
- `cena_jednostkowa` NUMBER(10,2) – cena **skopiowana** z produktu w chwili zakupu; `ck_pozycje_cena` CHECK (`>= 0`).
- `uk_pozycje_unik` UNIQUE (`id_zamowienia, id_produktu`) – ten sam produkt nie wystepuje dwa razy w jednym zamowieniu (zamiast tego zwieksza sie ilosc).

### Dlaczego kopiujemy cene?

Gdyby pozycja wskazywala tylko na produkt, pozniejsza zmiana ceny produktu zmienilaby wartosc juz zlozonych zamowien. Kopiujac cene do `cena_jednostkowa`, zachowujemy historie.

## 5.8. Tabela PLATNOSCI

```

131 -- platnosci
132 CREATE TABLE platnosci (
133 id_platnosci    NUMBER(10)      NOT NULL,
134 id_zamowienia   NUMBER(10)      NOT NULL,
135 kwota           NUMBER(12,2)    NOT NULL,
136 metoda         VARCHAR2(20)    NOT NULL,
137 status         VARCHAR2(20)    DEFAULT 'OCZEKUJACA' NOT NULL,
138 data_platnosci DATE            DEFAULT SYSDATE NOT NULL,
139
140 CONSTRAINT pk_platnosci PRIMARY KEY (id_platnosci),
141 CONSTRAINT fk_platnosci_zam FOREIGN KEY (id_zamowienia)
142 REFERENCES zamowienia (id_zamowienia),
143 CONSTRAINT ck_platnosci_metoda CHECK (metoda IN ('KARTA', 'PRZELEW', 'BLIK', 'POBRANIE')),
144 CONSTRAINT ck_platnosci_status CHECK (status IN ('OCZEKUJACA', 'ZREALIZOWANA', 'ODRZUCONA', 'ZWROCONA')),
145 CONSTRAINT ck_platnosci_kwota CHECK (kwota > 0)
146 );

```

- `id_platnosci` – PK; `id_zamowienia` + `fk_platnosci_zam` – FK.
- `kwota` NUMBER(12,2) + `ck_platnosci_kwota` CHECK (`kwota > 0`) – kwota dodatnia.
- `metoda` + `ck_platnosci_metoda` CHECK (... IN ('KARTA', 'PRZELEW', 'BLIK', 'POBRANIE')).
- `status` + `ck_platnosci_status` CHECK (... IN ('OCZEKUJACA', 'ZREALIZOWANA', 'ODRZUCONA', 'ZWROCONA')).

- `data_platnosci` `DATE` `DEFAULT` `SYSDATE`.

## 5.9. Tabela RECENZJE

```

149 -- recenzje produktow - jeden klient moze ocenic dany produkt tylko raz
150 CREATE TABLE recenzje (
151     id_recenzji    NUMBER(10)        NOT NULL,
152     id_produktu    NUMBER(10)        NOT NULL,
153     id_klienta     NUMBER(10)        NOT NULL,
154     ocena          NUMBER(1)         NOT NULL, -- 1-5 gwiazdek
155     tresc          VARCHAR2(1000),
156     data_dodania   DATE              DEFAULT SYSDATE NOT NULL,
157
158     CONSTRAINT pk_recenzje PRIMARY KEY (id_recenzji),
159     CONSTRAINT fk_recenzje_produkt FOREIGN KEY (id_produktu)
160         REFERENCES produkty (id_produktu) ON DELETE CASCADE,
161     CONSTRAINT fk_recenzje_klient FOREIGN KEY (id_klienta)
162         REFERENCES klienci (id_klienta),
163     CONSTRAINT ck_recenzje_ocena CHECK (ocena BETWEEN 1 AND 5),
164     CONSTRAINT uk_recenzje_unik UNIQUE (id_produktu, id_klienta)
165 );

```

- `id_recenzji` – PK.
- `id_produktu` + `fk_recenzje_produkt ... ON DELETE CASCADE` – recenzje znikaja z produktem.
- `id_klienta` + `fk_recenzje_klient` – autor recenzji.
- `ocena` `NUMBER(1)` + `ck_recenzje_ocena` `CHECK (ocena BETWEEN 1 AND 5)` – 1-5 gwiazdek.
- `tresc` `VARCHAR2(1000)` – tresc opinii.
- `uk_recenzje_unik` `UNIQUE (id_produktu, id_klienta)` – jeden klient ocenia produkt raz.

## 5.10. Tabela LOG\_OPERACJI

```

168 -- log do audytu - bedzie zapełniany triggerami
169 CREATE TABLE log_operacji (
170     id_logu        NUMBER(10)        NOT NULL,
171     uzytkownik     VARCHAR2(50)      DEFAULT USER NOT NULL,
172     operacja       VARCHAR2(10)      NOT NULL,
173     nazwa_tabeli   VARCHAR2(30)      NOT NULL,
174     id_rekordu     NUMBER(10),
175     data_op        TIMESTAMP         DEFAULT SYSTIMESTAMP NOT NULL,
176     szczegoly      VARCHAR2(500),
177
178     CONSTRAINT pk_log PRIMARY KEY (id_logu),
179     CONSTRAINT ck_log_operacja CHECK (operacja IN ('INSERT', 'UPDATE', 'DELETE'))
180 );

```

- `id_logu` – PK.
- `uzytkownik` `VARCHAR2(50)` `DEFAULT` `USER` – nazwa uzytkownika bazy (funkcja `USER`).
- `operacja` + `ck_log_operacja` `CHECK (... IN ('INSERT', 'UPDATE', 'DELETE'))`.
- `nazwa_tabeli` `VARCHAR2(30)`, `id_rekordu` `NUMBER(10)` – czego dotyczyła operacja.
- `data_op` `TIMESTAMP` `DEFAULT` `SYSTIMESTAMP` – dokładny czas operacji.
- `szczegoly` `VARCHAR2(500)` – opis tekstowy.

LOG\_OPERACJI nie ma kluczy obcych – przechowuje historie zdarzen niezaleznie od tego, czy rekord zrodlowy nadal istnieje. Wypelniaja go triggery audytowe (rozdzial 8).

## 5.11. Indeksy

```
183 -- indeksy na klucze obce i kolumny po ktorych czesto bedziemy szukac
184 CREATE INDEX idx_produkty_kategoria ON produkty (id_kategorii);
185 CREATE INDEX idx_zamowienia_klient ON zamowienia (id_klienta);
186 CREATE INDEX idx_zamowienia_status ON zamowienia (status);
187 CREATE INDEX idx_pozycje_zamowienie ON pozycje_zamowienia (id_zamowienia);
188 CREATE INDEX idx_pozycje_produkty ON pozycje_zamowienia (id_produkty);
189 CREATE INDEX idx_platnosci_zamowienie ON platnosci (id_zamowienia);
190 CREATE INDEX idx_recenzje_produkty ON recenzje (id_produkty);
191 CREATE INDEX idx_log_tabela ON log_operacji (nazwa_tabeli, data_op);
```

Indeksy przyspieszaja wyszukiwanie i laczenie tabel. Zalozone sa na kluczach obcych (np. `idx_produkty_kategoria`) oraz na kolumnach czesto filtrowanych (`idx_zamowienia_status`). Indeks `idx_log_tabela` jest zlozony (`nazwa_tabeli`, `data_op`) – przyspiesza przegladanie logu wg tabeli i czasu.

### Pojecie: indeks

Indeks to struktura przyspieszajaca wyszukiwanie wierszy po danej kolumnie (jak skorowidz w ksiazce). Zaklada sie go zwlaszcza na kolumnach uzywanych w warunkach WHERE i w zlaczeniach (klucze obce).

## 5.12. Triggery nadajace identyfikatory

```
194 -- triggery podstawiajace ID z sekwencji (zeby nie trzeba bylo go podawac w insercie)
195 CREATE OR REPLACE TRIGGER trg_klienci_bi
196 BEFORE INSERT ON klienci FOR EACH ROW
197 BEGIN
198   IF :NEW.id_klienta IS NULL THEN
199     :NEW.id_klienta := seq_klienci.NEXTVAL;
200   END IF;
201 END;
```

Dla kazdej z osmiu tabel istnieje analogiczny trigger. Powyzej przyklad dla KLIENCI:

- **Linia 195** – `CREATE OR REPLACE TRIGGER trg_klienci_bi` – nazwa (bi = before insert).
- **Linia 196** – `BEFORE INSERT ON klienci FOR EACH ROW` – uruchamiany przed wstawieniem, dla kazdego wiersza.
- **Linie 198-200** – jesli `:NEW.id_klienta IS NULL` (nie podano ID), pobierz `seq_klienci.NEXTVAL` i wpisz do `:NEW.id_klienta`.

### Pojecie: :NEW i trigger BEFORE INSERT

`:NEW` to rekord, ktory wlasnie ma byc wstawiony – w triggerze BEFORE mozna go zmodyfikowac. Tu uzupełniamy klucz glowny wartoscia z sekwencji. To standardowy sposob auto-inkrementacji w Oracle (sekwencja + trigger), uczony na wykladzie zamiast kolumn GENERATED AS IDENTITY.

## 6. Dane testowe - 02\_dane\_testowe.sql

Skrypt wypelnia baze spojnymi danymi. Uruchamiamy go po `projekt.sql`, a przed pakietami i triggerami – dzięki temu triggerzy audytowe nie loguja danych poczatkowych, a triggerzy magazynowe nie ingeruja w seed.

### 6.1. Czyszczenie danych

```
5 -- (w bazie trzymamy tylko hash bcrypt).
6
7 -- czyszczenie danych zeby skrypt byl idempotentny
8 DELETE FROM recenzje;
9 DELETE FROM platnosci;
10 DELETE FROM pozycje_zamowienia;
11 DELETE FROM zamowienia;
12 DELETE FROM produkty;
13 DELETE FROM kategorie;
14 DELETE FROM klienci;
15 DELETE FROM log_operacji;
16 COMMIT;
```

Najpierw `DELETE` ze wszystkich tabel w kolejnosci odwrotnej do zaleznosci (dzieci przed rodzicami), zeby nie naruszyc kluczy obcych: recenzje, platnosci, pozycje, zamowienia, produkty, kategorie, klienci, log. `COMMIT` zatwierdza usuniecie.

### 6.2. Wstawianie danych

Dane wstawiane sa w kolejnosci zgodnej z kluczami obcymi: kategorie (najpierw glowne, potem podkategorie), klienci, produkty, zamowienia, pozycje, platnosci, recenzje. Identyfikatory wpisane sa jawnie, by latwo powiazac rekordy. Przyklad wstawienia klienta (haslo to gotowy hash bcrypt):

```
36 INSERT INTO klienci (id_klienta, email, haslo_hash, imie, nazwisko, telefon, adres_ulica, adres_miasto, adres_
37 VALUES (4, 'marek.zielinski@example.com', '$2b$12$Q6.qf0sBaPzgCF.sTpNfN0mPPFmT4hpiids5j92aQ8uyZ0EA5i1/q', 'Mar
38 INSERT INTO klienci (id_klienta, email, haslo_hash, imie, nazwisko, telefon, adres_ulica, adres_miasto, adres_
```

- wszyscy klienci maja haslo `haslo123`, pracownik `praca123`, admin `admin123` (w bazie tylko hash),
- klient o id 5 jest nieaktywny (`aktywny='N'`) – do testu blokady logowania,
- produkty 4 i 11 maja stan 0, a produkt 11 jest nieaktywny – do testu niedostepnosci i ukrywania,
- zamowienia pokrywaja wszystkie piec statusow – zeby panel mial co pokazac.

### 6.3. Dociaganie sekwencji

```
108 -- dociagniecie sekwencji powyzej najwiekszego wstawionego ID,
109 -- zeby aplikacja generowala kolejne ID bez kolizji
110 DECLARE
111 PROCEDURE dobij(p_seq IN VARCHAR2, p_tab IN VARCHAR2, p_col IN VARCHAR2) IS
112     v_max NUMBER;
```

```

113     v_cur NUMBER;
114 BEGIN
115     EXECUTE IMMEDIATE 'SELECT NVL(MAX('||p_col||'),0) FROM '||p_tab INTO v_max;
116     LOOP
117         EXECUTE IMMEDIATE 'SELECT '||p_seq||'.NEXTVAL FROM dual' INTO v_cur;
118         EXIT WHEN v_cur >= v_max;
119     END LOOP;
120 END;
121 BEGIN
122     dobij('seq_klienci', 'klienci', 'id_klienta');
123     dobij('seq_kategorie', 'kategorie', 'id_kategorii');
124     dobij('seq_produkty', 'produkty', 'id_produkta');
125     dobij('seq_zamowienia', 'zamowienia', 'id_zamowienia');
126     dobij('seq_pozycje', 'pozycje_zamowienia', 'id_pozycji');
127     dobij('seq_platnosci', 'platnosci', 'id_platnosci');
128     dobij('seq_recenzje', 'recenzje', 'id_recenzji');
129     dobij('seq_log', 'log_operacji', 'id_logu');
130 END;
131 /

```

Ponieważ ID wpisano jawnie, sekwencje nadal stoja na 1 i kolidowałyby z danymi. Ten blok je “przewija”:

- **Linie 111-120** – zagniezdzona procedura `dobij(p_seq, p_tab, p_col)`: odczytuje `MAX(kolumna)` z tabeli (do `v_max`), a potem w petli pobiera `NEXTVAL` aż przekroczy to maksimum.
- **Linia 115** – `EXECUTE IMMEDIATE 'SELECT NVL(MAX('||p_col||'),0) FROM '||p_tab INTO v_max` – dynamiczne zapytanie liczące maksymalne ID (`NVL` daje 0 dla pustej tabeli).
- **Linie 116-119** – petla `LOOP ... EXIT WHEN v_cur >= v_max` – pobiera kolejne wartości sekwencji, aż osiągnie maksimum.
- **Linie 122-129** – wywołanie `dobij` dla każdej z ośmiu sekwencji.

Dzięki temu kolejne rekordy dodawane przez aplikację nie wejdą w konflikt z danymi testowymi.



## 7. Pakiety PL/SQL - 03\_pakiety.sql (omowienie pelne)

Pakiety zawieraja cala logike biznesowa sklepu. Styl zgodny z wykladem: prefiks `P_`, parametry z sufixem `_in`, typy przez `%TYPE`, slowo `AS` w ciecie, bledy przez `RAISE_APPLICATION_ERROR` z numerami -20001..-20006.

### Pojecie: pakiet (specyfikacja i ciało)

Pakiet sklada sie ze **specyfikacji** (`CREATE PACKAGE`) – deklaruje, co jest dostepne z zewnatrz (naglowki procedur i funkcji), oraz **ciala** (`CREATE PACKAGE BODY`) – zawiera implementacje. Aplikacja widzi tylko specyfikacje, co oddziela interfejs od szczegolow.

### 7.1. Specyfikacja pakietu P\_ZAMOWIENIA

```
18 CREATE OR REPLACE PACKAGE P_ZAMOWIENIA AS
19
20   -- tworzy nowe (puste) zamowienie w statusie NOWE, zwraca jego ID
21   FUNCTION zloz_zamowienie(id_klienta_in IN klienci.id_klienta%TYPE,
22                           adres_in      IN zamowienia.adres_dostawy%TYPE)
23       RETURN zamowienia.id_zamowienia%TYPE;
24
25   -- dodaje pozycje do zamowienia (kopiuje cene z produktu)
26   PROCEDURE dodaj_pozycje(id_zamowienia_in IN zamowienia.id_zamowienia%TYPE,
27                           id_produktu_in   IN produkty.id_produktu%TYPE,
28                           ilosc_in          IN pozycje_zamowienia.ilosc%TYPE);
29
30   -- zwraca sume zamowienia (suma ilosc * cena_jednostkowa)
31   FUNCTION oblicz_sume_zamowienia(id_zamowienia_in IN zamowienia.id_zamowienia%TYPE)
32       RETURN NUMBER;
33
34   -- przelicza i zapisuje kolumne suma w zamowieniu
35   PROCEDURE przelicz_sume(id_zamowienia_in IN zamowienia.id_zamowienia%TYPE);
36
37   -- zmienia status zamowienia z kontrola dozwolonych przejsc
38   PROCEDURE zmien_status(id_zamowienia_in IN zamowienia.id_zamowienia%TYPE,
39                           nowy_status_in  IN zamowienia.status%TYPE);
40
41   -- anuluje zamowienie (tylko ze statusu NOWE lub OPLACONE)
42   PROCEDURE anuluj_zamowienie(id_zamowienia_in IN zamowienia.id_zamowienia%TYPE);
43
44   -- rejestruje platnosc i ustawia zamowienie na OPLACONE
45   PROCEDURE dodaj_platnosc(id_zamowienia_in IN zamowienia.id_zamowienia%TYPE,
46                           metoda_in         IN platnosci.metoda%TYPE);
47
48 END P_ZAMOWIENIA;
```

- **Linia 18** – `CREATE OR REPLACE PACKAGE P_ZAMOWIENIA AS` – poczatek specyfikacji.
- **Linie 21-24** – `FUNCTION zloz_zamowienie(...) RETURN ...%TYPE` – funkcja tworzaca zamowienie, zwraca jego ID.
- **Linie 26-29** – `PROCEDURE dodaj_pozycje(...)` – dodaje pozycje do zamowienia.
- **Linie 31-36** – funkcja `oblicz_sume_zamowienia` i procedura `przelicz_sume`.
- **Linie 38-45** – `zmien_status`, `anuluj_zamowienie`, `dodaj_platnosc`.
- **Linia 48** – `END P_ZAMOWIENIA;` – koniec specyfikacji.

### Pojecie: %TYPE

Zapis `klienci.id_klienta%TYPE` oznacza "ten sam typ, co kolumna `id_klienta`". Dzięki temu parametr automatycznie pasuje do typu w bazie – gdy zmienimy typ kolumny, kod pakietu nie wymaga poprawek. To zalecana praktyka z wykładu.

## 7.2. Ciało pakietu - funkcja `zloz_zamowienie`

```
53 FUNCTION zloz_zamowienie(id_klienta_in IN klienci.id_klienta%TYPE,  
54                           adres_in      IN zamowienia.adres_dostawy%TYPE)  
55 RETURN zamowienia.id_zamowienia%TYPE  
56 AS  
57   v_aktywny  klienci.aktywny%TYPE;  
58   v_id        zamowienia.id_zamowienia%TYPE;  
59 BEGIN  
60   -- sprawdzenie czy klient istnieje i jest aktywny  
61   BEGIN  
62     SELECT aktywny INTO v_aktywny FROM klienci WHERE id_klienta = id_klienta_in;  
63   EXCEPTION  
64     WHEN NO_DATA_FOUND THEN  
65       RAISE_APPLICATION_ERROR(-20001, 'Klient nie istnieje.');66   END;  
67  
68   IF v_aktywny = 'N' THEN  
69     RAISE_APPLICATION_ERROR(-20001, 'Klient jest nieaktywny.');70   END IF;  
71  
72   INSERT INTO zamowienia (id_klienta, status, suma, adres_dostawy)  
73   VALUES (id_klienta_in, 'NOWE', 0, adres_in)  
74   RETURNING id_zamowienia INTO v_id;  
75  
76   RETURN v_id;  
77 END zloz_zamowienie;
```

- **Linie 53-55** – nagłówek funkcji powtórzony w ciele; `RETURN ...%TYPE` określa typ wyniku.
- **Linia 56 (AS)** – początek części deklaracyjnej.
- **Linie 57-58** – zmienne lokalne `v_aktywny` i `v_id` (prefiks `v_`).
- **Linie 61-66** – wewnętrzny blok pobiera kolumnę `aktywny` klienta; jeśli klienta nie ma, wyjątek `NO_DATA_FOUND` zostaje zamieniony na czytelny błąd -20001.
- **Linie 68-70** – jeśli klient jest nieaktywny ('N') – również błąd -20001.
- **Linie 72-74** – `INSERT ... RETURNING id_zamowienia INTO v_id` wstawia zamówienie w statusie `NOWE` i od razu odbiera nadane przez trigger ID.
- **Linia 76** – `RETURN v_id` – funkcja zwraca ID nowego zamówienia.

### Pojecie: `NO_DATA_FOUND` i `RETURNING INTO`

`SELECT ... INTO` zgłasza wyjątek `NO_DATA_FOUND`, gdy zapytanie nie zwróci wiersza – łapiemy go i zamieniamy na zrozumiały komunikat. `INSERT ... RETURNING kolumna INTO zmienna` pozwala od razu odczytać wartość nadaną przez trigger/sekwencję, bez osobnego `SELECT`.

## 7.3. Procedura dodaj\_pozycje

```
80  PROCEDURE dodaj_pozycje(id_zamowienia_in IN zamowienia.id_zamowienia%TYPE,  
81                        id_produktu_in   IN produkty.id_produktu%TYPE,  
82                        ilosc_in         IN pozycje_zamowienia.ilosc%TYPE)  
83  AS  
84    v_aktywny produkty.aktywny%TYPE;  
85    v_cena    produkty.cena%TYPE;  
86    v_stan    produkty.stan_magazynowy%TYPE;  
87  BEGIN  
88    -- pobranie danych produktu  
89    BEGIN  
90      SELECT aktywny, cena, stan_magazynowy  
91        INTO v_aktywny, v_cena, v_stan  
92        FROM produkty WHERE id_produktu = id_produktu_in;  
93    EXCEPTION  
94      WHEN NO_DATA_FOUND THEN  
95        RAISE_APPLICATION_ERROR(-20002, 'Produkt nie istnieje.');
```

```
96    END;  
97  
98    IF v_aktywny = 'N' THEN  
99      RAISE_APPLICATION_ERROR(-20002, 'Produkt jest nieaktywny.');
```

```
100  END IF;  
101  
102  IF ilosc_in > v_stan THEN  
103    RAISE_APPLICATION_ERROR(-20003, 'Brak wystarczajacej ilosci na stanie (dostepne: ' || v_stan || ').');
```

```
104  END IF;  
105  
106  -- jesli produkt jest juz w zamowieniu - zwiekszamy ilosc, inaczey dodajemy  
107  UPDATE pozycje_zamowienia  
108    SET ilosc = ilosc + ilosc_in  
109    WHERE id_zamowienia = id_zamowienia_in  
110    AND id_produktu = id_produktu_in;  
111  
112  IF SQL%ROWCOUNT = 0 THEN  
113    INSERT INTO pozycje_zamowienia (id_zamowienia, id_produktu, ilosc, cena_jednostkowa)  
114      VALUES (id_zamowienia_in, id_produktu_in, ilosc_in, v_cena);  
115  END IF;  
116  
117  przelicz_sume(id_zamowienia_in);  
118  END dodaj_pozycje;
```

- **Linie 84-86** – zmienne na dane produktu (aktywnosc, cena, stan).
- **Linie 89-96** – pobranie tych danych jednym SELECT-em; brak produktu → blad -20002.
- **Linie 98-100** – produkt nieaktywny → blad -20002.
- **Linie 102-104** – zadana ilosc wieksza niz stan → blad -20003 (z podaniem dostepnej ilosci).
- **Linie 107-110** – UPDATE pozycje\_zamowienia SET ilosc = ilosc + ilosc\_in WHERE ... – jesli produkt juz jest w zamowieniu, zwieksza ilosc.
- **Linie 112-115** – IF SQL%ROWCOUNT = 0 THEN INSERT ... – jesli UPDATE nic nie zmienil (produktu jeszcze nie bylo), wstawia nowa pozycje z cena skopiowana z produktu.
- **Linia 117** – przelicz\_sume(id\_zamowienia\_in) – aktualizuje sume zamowienia.

### Pojecie: SQL%ROWCOUNT

SQL%ROWCOUNT to atrybut kursora niejawnego – liczba wierszy zmienionych przez ostatnie polecenie DML. Tu sprawdzamy, czy UPDATE cos zaktualizowal; jesli 0 – trzeba zrobic INSERT. To wzorzec “UPDATE albo INSERT”.

## 7.4. Funkcje oblicz\_sume\_zamowienia i przelicz\_sume

```
121 FUNCTION oblicz_sume_zamowienia(id_zamowienia_in IN zamowienia.id_zamowienia%TYPE)
122 RETURN NUMBER
123 AS
124     v_suma NUMBER := 0;
125 BEGIN
126     SELECT NVL(SUM(ilosc * cena_jednostkowa), 0)
127     INTO v_suma
128     FROM pozycje_zamowienia
129     WHERE id_zamowienia = id_zamowienia_in;
130     RETURN v_suma;
131 END oblicz_sume_zamowienia;
132
133
134 PROCEDURE przelicz_sume(id_zamowienia_in IN zamowienia.id_zamowienia%TYPE)
135 AS
136 BEGIN
137     UPDATE zamowienia
138     SET suma = oblicz_sume_zamowienia(id_zamowienia_in)
139     WHERE id_zamowienia = id_zamowienia_in;
140 END przelicz_sume;
```

- **oblicz\_sume\_zamowienia** – `SELECT NVL(SUM(ilosc * cena_jednostkowa), 0) INTO v_suma` sumuje wartosc pozycji; `NVL(...,0)` zwraca 0, gdy zamowienie nie ma pozycji.
- **przelicz\_sume** – `UPDATE zamowienia SET suma = oblicz_sume_zamowienia(...)` zapisuje wynik funkcji do kolumny suma.

### Pojecie: funkcja vs procedura, NVL

Funkcja zwraca wartosc ( `RETURN` ) i mozna jej uzyc w zapytaniu lub wyrazeniu; procedura wykonuje akcje i nic nie zwraca. `NVL(x, y)` zwraca `y`, gdy `x` jest `NULL`.

## 7.5. Procedura zmien\_status (maszyna stanow)

```
143 PROCEDURE zmien_status(id_zamowienia_in IN zamowienia.id_zamowienia%TYPE,
144                        nowy_status_in IN zamowienia.status%TYPE)
145 AS
146     v_obecny zamowienia.status%TYPE;
147     v_ok      BOOLEAN := FALSE;
148 BEGIN
149     BEGIN
150         SELECT status INTO v_obecny FROM zamowienia WHERE id_zamowienia = id_zamowienia_in;
151     EXCEPTION
152         WHEN NO_DATA_FOUND THEN
153             RAISE_APPLICATION_ERROR(-20005, 'Zamowienie nie istnieje.');
```

154 END;

155

156 -- dozwolone przejścia statusow

157 IF v\_obecny = 'NOWE' AND nowy\_status\_in IN ('OPLACONE', 'ANULOWANE') THEN v\_ok := TRUE;

158 ELSIF v\_obecny = 'OPLACONE' AND nowy\_status\_in IN ('WYSLANE', 'ANULOWANE') THEN v\_ok := TRUE;

159 ELSIF v\_obecny = 'WYSLANE' AND nowy\_status\_in = 'DOSTARCZONE' THEN v\_ok := TRUE;

160 END IF;

161

162 IF NOT v\_ok THEN

163 RAISE\_APPLICATION\_ERROR(-20004, 'Niedozwolona zmiana statusu: ' || v\_obecny || ' -> ' || nowy\_status\_in);

164 END IF;

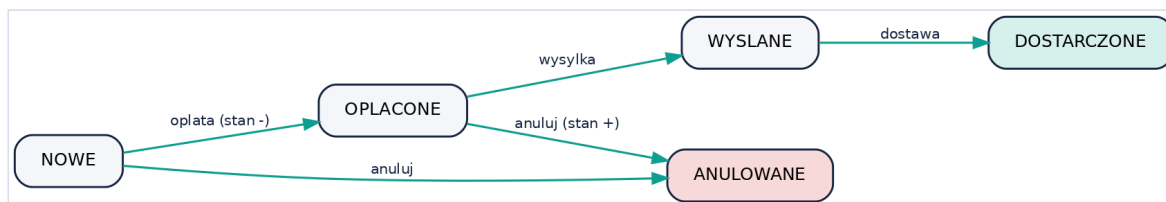
165

```

166 UPDATE zamowienia SET status = nowy_status_in WHERE id_zamowienia = id_zamowienia_in;
167 END zmien_status;

```

- **Linie 149-154** – pobranie bieżącego statusu; brak zamówienia → błąd -20005.
- **Linie 157-160** – seria warunków IF/ELSIF ustawia `v_ok := TRUE` tylko dla dozwolonych przejść: NOWE→OPLACONE/ANULOWANE, OPLACONE→WYSLANE/ANULOWANE, WYSLANE→DOSTARCZONE.
- **Linie 162-164** – jeśli przejście niedozwolone (`NOT v_ok`) → błąd -20004.
- **Linia 166** – dopiero teraz `UPDATE zamowienia SET status = ...`, co uruchamia trigger magazynowy (rozdział 8).



Rysunek 7.1. Dozwolone przejścia statusów zamówienia.

## 7.6. anuluj\_zamowienie i dodaj\_platnosc

```

170 PROCEDURE anuluj_zamowienie(id_zamowienia_in IN zamowienia.id_zamowienia%TYPE)
171 AS
172 BEGIN
173     -- korzysta z walidacji w zmien_status (z NOWE/OPLACONE można anulować)
174     zmien_status(id_zamowienia_in, 'ANULOWANE');
175 END anuluj_zamowienie;
176
177
178 PROCEDURE dodaj_platnosc(id_zamowienia_in IN zamowienia.id_zamowienia%TYPE,
179                          metoda_in         IN platnosci.metoda%TYPE)
180 AS
181     v_suma zamowienia.suma%TYPE;
182 BEGIN
183     SELECT suma INTO v_suma FROM zamowienia WHERE id_zamowienia = id_zamowienia_in;
184
185     INSERT INTO platnosci (id_zamowienia, kwota, metoda, status)
186     VALUES (id_zamowienia_in, v_suma, metoda_in, 'ZREALIZOWANA');
187
188     -- zmiana statusu na OPLACONE uruchomi trigger zdejmujący stan magazynowy
189     zmien_status(id_zamowienia_in, 'OPLACONE');
190 EXCEPTION
191     WHEN NO_DATA_FOUND THEN
192         RAISE_APPLICATION_ERROR(-20005, 'Zamowienie nie istnieje.');
```

- **anuluj\_zamowienie** – wywołuje `zmien_status(..., 'ANULOWANE')`, korzystając z tej samej walidacji (anulować można tylko z NOWE lub OPLACONE).
- **dodaj\_platnosc** – odczytuje sumę zamówienia, wstawia rekord do PLATNOSCI ze statusem ZREALIZOWANA, a następnie `zmien_status(..., 'OPLACONE')` – co uruchamia trigger zdejmujący towar z magazynu. Blok obsługuje `NO_DATA_FOUND` (brak zamówienia → -20005).

## 7.7. Pakiet P\_KLIENCI - rejestracja

```
202 CREATE OR REPLACE PACKAGE P_KLIENCI AS
203
204   -- rejestruje nowego klienta; haslo przychodzi juz zahaszowane (bcrypt) z aplikacji
205   FUNCTION rejestruj(email_in      IN klienci.email%TYPE,
206                      haslo_hash_in IN klienci.haslo_hash%TYPE,
207                      imie_in       IN klienci.imie%TYPE,
208                      nazwisko_in   IN klienci.nazwisko%TYPE,
209                      telefon_in    IN klienci.telefon%TYPE DEFAULT NULL,
210                      ulica_in      IN klienci.adres_ulica%TYPE DEFAULT NULL,
211                      miasto_in     IN klienci.adres_miasto%TYPE DEFAULT NULL,
212                      kod_in        IN klienci.adres_kod%TYPE DEFAULT NULL)
213     RETURN klienci.id_klienta%TYPE;
214
215 END P_KLIENCI;
216 /
217
218 CREATE OR REPLACE PACKAGE BODY P_KLIENCI AS
219
220   FUNCTION rejestruj(email_in      IN klienci.email%TYPE,
221                      haslo_hash_in IN klienci.haslo_hash%TYPE,
222                      imie_in       IN klienci.imie%TYPE,
223                      nazwisko_in   IN klienci.nazwisko%TYPE,
224                      telefon_in    IN klienci.telefon%TYPE DEFAULT NULL,
225                      ulica_in      IN klienci.adres_ulica%TYPE DEFAULT NULL,
226                      miasto_in     IN klienci.adres_miasto%TYPE DEFAULT NULL,
227                      kod_in        IN klienci.adres_kod%TYPE DEFAULT NULL)
228     RETURN klienci.id_klienta%TYPE
229   AS
230     v_ile NUMBER;
231     v_id  klienci.id_klienta%TYPE;
232   BEGIN
233     SELECT COUNT(*) INTO v_ile FROM klienci WHERE LOWER(email) = LOWER(email_in);
234     IF v_ile > 0 THEN
235       RAISE_APPLICATION_ERROR(-20006, 'Email jest juz zajety.');
```

- **Linie 202-216** – specyfikacja: funkcja `rejestruj` z parametrami konta; parametry adresowe maja `DEFAULT NULL` (sa opcjonalne).
- **Linie 233-236** – `SELECT COUNT(*) ... WHERE LOWER(email)=LOWER(email_in)` – sprawdza, czy email jest juz zajety (bez wzgledu na wielkosc liter); jesli tak → blad -20006.
- **Linie 238-242** – `INSERT` nowego klienta z rola `KLIENT`; haslo przychodzi **juz zahashowane** (bcrypt liczony w aplikacji), wiec baza nigdy nie widzi hasla jawnego.
- **Linia 244** – `RETURN v_id` zwraca ID nowego klienta.

## 7.8. Obsluga bladow w pakietach

| Numer  | Znaczenie                               |
|--------|-----------------------------------------|
| -20001 | klient nie istnieje lub jest nieaktywny |

| Numer  | Znaczenie                                |
|--------|------------------------------------------|
| -20002 | produkt nie istnieje lub jest nieaktywny |
| -20003 | brak wystarczającej ilości na stanie     |
| -20004 | niedozwolona zmiana statusu              |
| -20005 | zamowienie nie istnieje                  |
| -20006 | email już zajęty                         |

**Pojecie: RAISE\_APPLICATION\_ERROR**

`RAISE_APPLICATION_ERROR(numer, komunikat)` zgłasza własny błąd z numerem z zakresu -20000...-20999 i tekstem. Aplikacja odczytuje numer i komunikat, by pokazać użytkownikowi czytelna informację (np. "Brak na stanie"). Świadomie nie użyto `PRAGMA EXCEPTION_INIT`, której nie było na zajęciach.

## 8. Triggery - 04\_triggery.sql (omowienie pelne)

Triggery realizuja dwa zadania: kontrole magazynu (pozycje 8.1–8.2) oraz audyt zmian (8.3). Uruchamiamy je po pakietach.

### Pojecie: trigger, BEFORE/AFTER, FOR EACH ROW

Trigger to kod uruchamiany automatycznie przy operacji na tabeli. **BEFORE** dziala przed zapisem (mozna zmodyfikowac dane lub przerwac operacje), **AFTER** po zapisie (np. zareagowac w innej tabeli). **FOR EACH ROW** oznacza wykonanie dla kazdego wiersza. **:NEW** to nowa wartosc wiersza, **:OLD** – poprzednia.

### 8.1. Kontrola stanu przy dodawaniu pozycji

```
11 CREATE OR REPLACE TRIGGER trg_pozycje_stan
12 BEFORE INSERT OR UPDATE ON pozycje_zamowienia
13 FOR EACH ROW
14 DECLARE
15   v_stan produkty.stan_magazynowy%TYPE;
16 BEGIN
17   SELECT stan_magazynowy INTO v_stan
18     FROM produkty WHERE id_produktu = :NEW.id_produktu;
19
20   IF :NEW.ilosc > v_stan THEN
21     RAISE_APPLICATION_ERROR(-20003,
22       'Brak wystarczajacej ilosci na stanie (dostepne: ' || v_stan || ').');
23   END IF;
24 END;
```

- **Linia 11-13** – `trg_pozycje_stan BEFORE INSERT OR UPDATE ON pozycje_zamowienia FOR EACH ROW` – uruchamiany przed dodaniem/zmiana pozycji.
- **Linia 14-15** – deklaracja zmiennej `v_stan` na stan produktu.
- **Linia 17-18** – `SELECT stan_magazynowy INTO v_stan FROM produkty WHERE id_produktu = :NEW.id_produktu` – pobiera aktualny stan dodawanego produktu.
- **Linie 20-23** – jesli `:NEW.ilosc > v_stan`, zgłasza blad -20003. To druga warstwa ochrony (obok walidacji w pakiecie) – chroni nawet przy bezposrednim INSERT do tabeli.

### 8.2. Zdejmowanie i zwrot stanu magazynowego

```
33 CREATE OR REPLACE TRIGGER trg_zam_magazyn
34 AFTER UPDATE OF status ON zamowienia
35 FOR EACH ROW
36 DECLARE
37   v_stan produkty.stan_magazynowy%TYPE;
38 BEGIN
39   -- oplacenie -> zdejmij stan
40   IF :NEW.status = 'OPLACONE' AND :OLD.status <> 'OPLACONE' THEN
41     FOR poz IN (SELECT id_produktu, ilosc FROM pozycje_zamowienia
42                 WHERE id_zamowienia = :NEW.id_zamowienia) LOOP
43       -- kontrola zeby nie zejsc ponizej zera (czytelny blad zamiast ORA-02290)
44       SELECT stan_magazynowy INTO v_stan FROM produkty WHERE id_produktu = poz.id_produktu;
45       IF v_stan < poz.ilosc THEN
```



```

46      RAISE_APPLICATION_ERROR(-20003,
47      'Brak wystarczającej ilości na stanie przy opłacaniu (produkt ' || poz.id_produktu || ').');
48  END IF;
49  UPDATE produkty
50  SET stan_magazynowy = stan_magazynowy - poz.ilosc
51  WHERE id_produktu = poz.id_produktu;
52  END LOOP;
53
54  -- anulowanie opłaconego -> zwroc stan
55  ELSIF :NEW.status = 'ANULOWANE' AND :OLD.status = 'OPLACONE' THEN
56  FOR poz IN (SELECT id_produktu, ilosc FROM pozycje_zamowienia
57              WHERE id_zamowienia = :NEW.id_zamowienia) LOOP
58      UPDATE produkty
59      SET stan_magazynowy = stan_magazynowy + poz.ilosc
60      WHERE id_produktu = poz.id_produktu;
61  END LOOP;
62  END IF;
63 END;

```

- **Linia 34-35** – `trg_zam_magazyn` AFTER UPDATE OF status ON zamowienia FOR EACH ROW – reaguje po zmianie kolumny status.
- **Linie 40-52** – gdy status zmienia się na OPLACONE (a wcześniej nie był): petla `FOR poz IN (SELECT ... FROM pozycje_zamowienia WHERE id_zamowienia = :NEW.id_zamowienia)` przechodzi po pozycjach. Dla każdej pobiera stan; jeśli za mało → czytelny błąd -20003; w przeciwnym razie `UPDATE produkty SET stan_magazynowy = stan_magazynowy - poz.ilosc`.
- **Linie 54-61** – gdy opłacone zamówienie jest anulowane (OPLACONE→ANULOWANE): analogiczna petla **zwraca** towar (`stan_magazynowy + poz.ilosc`).

#### Pojecie: kursor FOR LOOP

`FOR rekord IN (SELECT ...) LOOP ... END LOOP;` to petla po wynikach zapytania – w każdym obrocie zmienna `poz` przyjmuje kolejny wiersz. Wygodny sposób iteracji bez ręcznego otwierania kursora.

#### Dlaczego nie ma problemu mutating table?

Problem “mutating table” pojawia się, gdy trigger wierszowy czyta lub zmienia tabelę, na której wisi. Tu trigger wisi na ZAMOWIENIA, a modyfikuje PRODUKTY i czyta POZYCJE\_ZAMOWIENIA – to inne tabele, więc problem nie występuje. Podobnie triggerzy audytowe piszą do osobnej tabeli LOG\_OPERACJI.

## 8.3. Triggerzy audytowe

Dla tabel KLIENCI, PRODUKTY, ZAMOWIENIA i PLATNOSCI istnieje analogiczny trigger audytowy. Poniżej przykład dla PRODUKTY:

```

89 CREATE OR REPLACE TRIGGER trg_log_produkty
90 AFTER INSERT OR UPDATE OR DELETE ON produkty
91 FOR EACH ROW
92 DECLARE
93   v_op log_operacji.operacja%TYPE;
94   v_id log_operacji.id_rekordu%TYPE;
95   v_szc log_operacji.szczegoly%TYPE;
96 BEGIN
97   IF INSERTING THEN
98     v_op := 'INSERT'; v_id := :NEW.id_produktu;
99     v_szc := 'Dodano produkt: ' || :NEW.nazwa;
100  ELSIF UPDATING THEN

```

```

101     v_op := 'UPDATE'; v_id := :NEW.id_produktu;
102     v_szc := 'Zmiana produktu: ' || :NEW.nazwa || ' (stan: ' || :NEW.stan_magazynowy || ')';
103 ELSE
104     v_op := 'DELETE'; v_id := :OLD.id_produktu;
105     v_szc := 'Usunieto produkt: ' || :OLD.nazwa;
106 END IF;
107
108 INSERT INTO log_operacji (operacja, nazwa_tabeli, id_rekordu, szczegoly)
109 VALUES (v_op, 'PRODUKTY', v_id, v_szc);
110 END;

```

- **Linia 89-91** – AFTER INSERT OR UPDATE OR DELETE ON produkty FOR EACH ROW – reaguje na kazda zmianę.
- **Linie 92-95** – zmienne na rodzaj operacji, ID rekordu i opis.
- **Linie 97-106** – IF INSERTING ... ELSIF UPDATING ... ELSE (DELETING) – pseudokolumny rozpoznają rodzaj operacji; dla INSERT/UPDATE biera ID z :NEW, dla DELETE z :OLD, i budują opis.
- **Linie 108-109** –  
INSERT INTO log\_operacji (operacja, nazwa\_tabeli, id\_rekordu, szczegoly) VALUES (...).  
Kolumny uzytkownik i data\_op wypelniaja sie same (DEFAULT USER i SYSTIMESTAMP).

#### Pojecie: INSERTING / UPDATING / DELETING

To pseudokolumny dostępne w triggerze łączącym kilka operacji ( INSERT OR UPDATE OR DELETE ). Pozwalają w jednym triggerze rozpoznać, która operacja go wywołała, i zachować się odpowiednio.

Triggery dla KLIENCI, ZAMOWIENIA i PLATNOSCI (linie 72-86, 113-134, 137-158) działają tak samo, różnią się tylko nazwa tabeli i treścią opisu (np. dla ZAMOWIENIA opis zawiera zmianę statusu :OLD.status -> :NEW.status).

## 9. Role i uprawnienia - 05\_role.sql (omowienie pelne)

Bezpieczenstwo realizujemy zgodnie z wykladem: przez role i widoki (bez kontekstow/VPD). Skrypt uruchamiamy jako wlasciciel schematu z przywilejami CREATE ROLE i CREATE USER.

### Pojecie: widok (VIEW)

Widok to zapisane zapytanie, ktore zachowuje sie jak wirtualna tabela. Sluzy do ukrywania danych (np. tylko aktywne produkty) i upraszczania zapytan. Klient korzysta z widokow zamiast z tabel.

### 9.1. Widoki

```
25 CREATE OR REPLACE VIEW v_katalog AS
26 SELECT p.id_produktu,
27        p.nazwa,
28        p.opis,
29        p.cena,
30        p.stan_magazynowy,
31        k.nazwa AS kategoria,
32        p.id_kategorii
33 FROM produkty p
34 JOIN kategorie k ON k.id_kategorii = p.id_kategorii
35 WHERE p.aktyny = 'T';
36
37 -- zamowienia z danymi klienta - dla personelu
38 CREATE OR REPLACE VIEW v_zamowienia_pelne AS
39 SELECT z.id_zamowienia,
40        z.data_zamowienia,
41        z.status,
42        z.suma,
43        z.adres_dostawy,
44        k.id_klienta,
45        k.imie || ' ' || k.nazwisko AS klient
46 FROM zamowienia z
47 JOIN klienci k ON k.id_klienta = z.id_klienta;
```

- **v\_katalog (25-35)** - laczy PRODUKTY z KATEGORIE i pokazuje tylko produkty `aktyny = 'T'`; to widzi klient w katalogu (zamiast pelnej tabeli PRODUKTY).
- **v\_zamowienia\_pelne (37-47)** - zamowienia z dolaczonym imieniem i nazwiskiem klienta (`k.imie || ' ' || k.nazwisko AS klient`); uzywany w panelu.

### 9.2. Sprzatanie i tworzenie rol

```
51 -- SPRZATANIE (zeby skrypt byl idempotentny) - kazdy DROP w osobnym bloku,
52 -- bledy ignorujemy (gdy obiekt jeszcze nie istnieje)
53 -- =====
54 BEGIN EXECUTE IMMEDIATE 'DROP USER app_klient CASCADE'; EXCEPTION WHEN OTHERS THEN NULL; END;
55 /
56 BEGIN EXECUTE IMMEDIATE 'DROP USER app_pracownik CASCADE'; EXCEPTION WHEN OTHERS THEN NULL; END;
57 /
58 BEGIN EXECUTE IMMEDIATE 'DROP USER app_admin CASCADE'; EXCEPTION WHEN OTHERS THEN NULL; END;
59 /
```

```

60 BEGIN EXECUTE IMMEDIATE 'DROP ROLE rola_klient'; EXCEPTION WHEN OTHERS THEN NULL; END;
61 /
62 BEGIN EXECUTE IMMEDIATE 'DROP ROLE rola_pracownik'; EXCEPTION WHEN OTHERS THEN NULL; END;
63 /
64 BEGIN EXECUTE IMMEDIATE 'DROP ROLE rola_admin'; EXCEPTION WHEN OTHERS THEN NULL; END;
65 /
66
67
68 -- =====
69 -- ROLE
70 -- =====
71 CREATE ROLE rola_klient;
72 CREATE ROLE rola_pracownik;
73 CREATE ROLE rola_admin;

```

- **Linie 54-64** – każdy DROP USER/ROLE w osobnym bloku z EXCEPTION WHEN OTHERS THEN NULL – jeśli obiekt nie istnieje, błąd jest ignorowany. Dzięki temu skrypt jest idempotentny (można go odpalać wielokrotnie).
- **Linie 71-73** – CREATE ROLE rola\_klient / rola\_pracownik / rola\_admin.

### Pojęcie: rola, uprawnienia systemowe i obiektowe

Rola to nazwany zbiór uprawnień, który nadaje się użytkownikom. Uprawnienia systemowe pozwalają na czynności w bazie (np. CREATE SESSION – logowanie). Uprawnienia obiektowe dotyczą konkretnych obiektów (np. SELECT na widoku, EXECUTE na pakiecie).

## 9.3. Nadawanie uprawnień

```

79 GRANT SELECT ON v_katalog TO rola_klient;
80 GRANT SELECT ON kategorie TO rola_klient;
81 GRANT SELECT ON recenzje TO rola_klient;
82 GRANT EXECUTE ON P_ZAMOWIENIA TO rola_klient;
83 GRANT EXECUTE ON P_KLIENCI TO rola_klient;
84
85 -- ----- uprawnienia roli PRACOWNIK -----
86 -- pracownik ma wszystko co klient + obsługa zamówień i podgląd produktów
87 GRANT rola_klient TO rola_pracownik;
88 GRANT SELECT, UPDATE ON produkty TO rola_pracownik;
89 GRANT SELECT ON zamówienia TO rola_pracownik;
90 GRANT SELECT ON pozycje_zamowienia TO rola_pracownik;
91 GRANT SELECT ON płatności TO rola_pracownik;
92 GRANT SELECT ON klienci TO rola_pracownik;
93 GRANT SELECT ON v_zamowienia_pelne TO rola_pracownik;
94
95 -- ----- uprawnienia roli ADMIN -----
96 -- admin ma wszystko co pracownik + zarządzanie produktami/kategoriami,
97 -- kontami klientów i podgląd logu operacji
98 GRANT rola_pracownik TO rola_admin;
99 GRANT INSERT, UPDATE, DELETE ON produkty TO rola_admin;
100 GRANT INSERT, UPDATE, DELETE ON kategorie TO rola_admin;
101 GRANT UPDATE, DELETE ON klienci TO rola_admin;
102 GRANT SELECT ON log_operacji TO rola_admin;

```

- **rola\_klient (79-83)** – SELECT na v\_katalog, kategoriach i recenzjach oraz EXECUTE na pakietach P\_ZAMOWIENIA i P\_KLIENCI. Klient nie ma dostępu do tabel wprost – tylko przez pakiety i widoki.

- **rola\_pracownik (87-93)** – `GRANT rola_klient TO rola_pracownik` (dziedziczenie) plus `SELECT, UPDATE` na produktach i `SELECT` na zamówieniach, pozycjach, płatnościach, klientach oraz widoku personelu.
- **rola\_admin (98-102)** – dziedziczy pracownika i dodatkowo `INSERT/UPDATE/DELETE` na produktach i kategoriach, `UPDATE/DELETE` na klientach oraz `SELECT` na `LOG_OPERACJI`.

Role tworzą hierarchie: admin zawiera uprawnienia pracownika, a pracownik – klienta. To wygodny i czytelny model “coraz więcej uprawnień”.

## 9.4. Użytkownicy bazy

```

106 -- UZYTKOWNICY (wymaga uprawnień DBA - patrz uwaga na gorze pliku)
107 -- Każdy użytkownik dostaje prawo logowania i jedną rolę.
108 -- =====
109 CREATE USER app_klient IDENTIFIED BY klient123;
110 CREATE USER app_pracownik IDENTIFIED BY prac123;
111 CREATE USER app_admin IDENTIFIED BY admin123;
112
113 GRANT CREATE SESSION TO app_klient;
114 GRANT CREATE SESSION TO app_pracownik;
115 GRANT CREATE SESSION TO app_admin;
116
117 GRANT rola_klient TO app_klient;
118 GRANT rola_pracownik TO app_pracownik;
119 GRANT rola_admin TO app_admin;

```

- **Linie 109-111** – `CREATE USER app_klient/app_pracownik/app_admin IDENTIFIED BY ...`.
- **Linie 113-115** – `GRANT CREATE SESSION` – prawo logowania.
- **Linie 117-119** – przypisanie roli do użytkownika.

Użytkownicy pokazują model uprawnień – logując się nimi w SQL Developer można zademonstrować, że np. `app_klient` nie może zmodyfikować produktów. Aplikacja Flask łączy się jako właściciel schematu, a role w UI rozróżnia kolumna `KLIENCI.rola`.

## 10. Testy - 90\_testy.sql (omowienie pelne)

Skrypt sprawdza logike i wypisuje wynik przez `DBMS_OUTPUT`. Na koncu `ROLLBACK` cofa zmiany testowe.

### Pojecie: `DBMS_OUTPUT`, `SQLCODE/SQLERRM`, `ROLLBACK`

`DBMS_OUTPUT.PUT_LINE` wypisuje tekst (wymaga `SET SERVEROUTPUT ON`). `SQLCODE` to numer ostatniego bledu, `SQLERRM` - jego tresc. `ROLLBACK` wycofuje niezatwierdzone zmiany - testy nie psuja danych.

### 10.1. Test 1 i 2 - magazyn po oplaceniu i anulowaniu

```
8 -- TEST 1: zloz zamowienie -> dodaj pozycje -> oplac -> stan ma spasc
9 -- =====
10 DECLARE
11   v_zam   zamowienia.id_zamowienia%TYPE;
12   v_przed produkty.stan_magazynowy%TYPE;
13   v_po    produkty.stan_magazynowy%TYPE;
14 BEGIN
15   SELECT stan_magazynowy INTO v_przed FROM produkty WHERE id_produktu = 7;
16
17   v_zam := P_ZAMOWIENIA.zloz_zamowienie(1, 'ul. Testowa 1, Warszawa');
18   P_ZAMOWIENIA.dodaj_pozycje(v_zam, 7, 2); -- 2 sztuki produktu 7
19   P_ZAMOWIENIA.dodaj_platnosc(v_zam, 'KARTA'); -- status OPLACONE -> trigger zdejmujze stan
20
21   SELECT stan_magazynowy INTO v_po FROM produkty WHERE id_produktu = 7;
22
23   IF v_po = v_przed - 2 THEN
24     DBMS_OUTPUT.PUT_LINE('TEST 1 OK: stan ' || v_przed || ' -> ' || v_po || ' (spadl o 2)');
25   ELSE
26     DBMS_OUTPUT.PUT_LINE('TEST 1 BLAD: stan ' || v_przed || ' -> ' || v_po);
27   END IF;
28
29 -- TEST 2: anuluj oplacone -> stan ma wrocic
30 P_ZAMOWIENIA.anuluj_zamowienie(v_zam);
31 SELECT stan_magazynowy INTO v_po FROM produkty WHERE id_produktu = 7;
32 IF v_po = v_przed THEN
33   DBMS_OUTPUT.PUT_LINE('TEST 2 OK: po anulowaniu stan wrocil do ' || v_po);
34 ELSE
35   DBMS_OUTPUT.PUT_LINE('TEST 2 BLAD: stan po anulowaniu = ' || v_po);
36 END IF;
37 EXCEPTION
38   WHEN OTHERS THEN
39     DBMS_OUTPUT.PUT_LINE('TEST 1/2 BLAD: ' || SQLERRM);
40 END;
41 /
```

- **Linia 15** - zapamietuje stan produktu 7 przed operacja (`v_przed`).
- **Linie 17-19** - sklada zamowienie, dodaje 2 sztuki, oplaca; oplacenie uruchamia trigger zdejmujacy stan.
- **Linie 23-27** - sprawdza, czy `v_po = v_przed - 2` i wypisuje OK lub BLAD.
- **Linie 30-36** (Test 2) - anuluje zamowienie i sprawdza, czy stan wrocil do wartosci poczatkowej.
- **Linie 37-39** - `EXCEPTION WHEN OTHERS` wypisuje ewentualny blad zamiast cichego przerwania.

## 10.2. Test 3 - przekroczenie stanu

```
45 -- TEST 3: proba dodania wiecej niz na stanie -> wyjatek -20003
46 -- =====
47 DECLARE
48   v_zam zamowienia.id_zamowienia%TYPE;
49 BEGIN
50   v_zam := P_ZAMOWIENIA.zloz_zamowienie(1, 'ul. Testowa 2');
51   P_ZAMOWIENIA.dodaj_pozycje(v_zam, 7, 99999); -- za duzo
52   DBMS_OUTPUT.PUT_LINE('TEST 3 BLAD: nie zglosil wyjatku');
53 EXCEPTION
54   WHEN OTHERS THEN
55     IF SQLCODE = -20003 THEN
56       DBMS_OUTPUT.PUT_LINE('TEST 3 OK: zgloszono wyjatek braku na stanie (' || SQLERRM || ')');
57     ELSE
58       DBMS_OUTPUT.PUT_LINE('TEST 3 BLAD: inny wyjatek: ' || SQLERRM);
59     END IF;
60 END;
61 /
```

Proba dodania 99999 sztuk. Oczekiwany wyjatek -20003. Blok `EXCEPTION` sprawdza `SQLCODE = -20003` i wypisuje OK; inny blad – BLAD.

## 10.3. Test 4 - niedozwolona zmiana statusu

```
65 -- TEST 4: niedozwolona zmiana statusu (NOWE -> DOSTARCZONE) -> wyjatek -20004
66 -- =====
67 DECLARE
68   v_zam zamowienia.id_zamowienia%TYPE;
69 BEGIN
70   v_zam := P_ZAMOWIENIA.zloz_zamowienie(1, 'ul. Testowa 3');
71   P_ZAMOWIENIA.zmien_status(v_zam, 'DOSTARCZONE'); -- z NOWE nie wolno
72   DBMS_OUTPUT.PUT_LINE('TEST 4 BLAD: nie zglosil wyjatku');
73 EXCEPTION
74   WHEN OTHERS THEN
75     IF SQLCODE = -20004 THEN
76       DBMS_OUTPUT.PUT_LINE('TEST 4 OK: zablokowano zla zmiane statusu (' || SQLERRM || ')');
77     ELSE
78       DBMS_OUTPUT.PUT_LINE('TEST 4 BLAD: inny wyjatek: ' || SQLERRM);
79     END IF;
80 END;
81 /
```

Proba przejścia `NOWE→DOSTARCZONE` (z pominięciem etapów). Oczekiwany wyjatek -20004, sprawdzany przez `SQLCODE`.

## 10.4. Test 5 - trigger audytowy

```
85 -- TEST 5: czy trigger audytowy faktycznie dopisuje wpis do LOG_OPERACJI
86 -- (porownanie liczby wpisow przed i po operacji UPDATE)
87 -- =====
88 DECLARE
89   v_przed NUMBER;
90   v_po     NUMBER;
91 BEGIN
92   SELECT COUNT(*) INTO v_przed FROM log_operacji;
93   -- dowolny UPDATE na produkcie uruchamia trigger trg_log_produkty
94   UPDATE produkty SET stan_magazynowy = stan_magazynowy WHERE id_produktu = 7;
95   SELECT COUNT(*) INTO v_po FROM log_operacji;
96
97   IF v_po > v_przed THEN
```

```

98     DBMS_OUTPUT.PUT_LINE('TEST 5 OK: trigger audytowy dopisał wpis (' || v_przed || ' -> ' || v_po || ')');
99     ELSE
100     DBMS_OUTPUT.PUT_LINE('TEST 5 BLAD: log nie urosł po operacji');
101     END IF;
102 EXCEPTION
103     WHEN OTHERS THEN
104     DBMS_OUTPUT.PUT_LINE('TEST 5 BLAD: ' || SQLERRM);
105 END;
106 /

```

Test liczy wpisy w LOG\_OPERACJI przed i po operacji UPDATE na produkcie. Jeśli po operacji wpisów jest więcej (`v_po > v_przed`), trigger audytowy działa – OK. Na końcu skryptu ROLLBACK cofa wszystkie zmiany testowe.



# 11. Aplikacja kliencka (Flask)

Aplikacja jest cienka – deleguje logikę do bazy. Operacje zapisu wywołują pakiety PL/SQL (`callproc/callfunc`), odczyty to SELECT-y, często z widoków.

## 11.1. Konfiguracja - config.py

```
1 # Konfiguracja połączenia z baza Oracle.
2 # Aplikacja łączy się jako właściciel schematu (ten sam użytkownik, w którym
3 # odpaliles projekt.sql i reszta skryptów).
4 # Można nadpisać przez zmienne środowiskowe albo zmienić wartości poniżej.
5
6 import os
7
8 # dane logowania do bazy - ZMIEN na swoje
9 DB_USER = os.environ.get("DB_USER", "sklep")          # np. twój użytkownik schematu
10 DB_PASSWORD = os.environ.get("DB_PASSWORD", "sklep")  # hasło do bazy
11
12 # DSN: host:port/service_name (typowo dla Oracle 19c XE -> XEPDB1)
13 DB_DSN = os.environ.get("DB_DSN", "localhost:1521/XEPDB1")
14
15 # klucz sesji Flaska
16 SECRET_KEY = os.environ.get("SECRET_KEY", "tajny-klucz-dev-zmienic")
```

Dane połączenia (użytkownik, hasło, DSN) można nadpisać zmiennymi środowiskowymi; DSN ma format `host:port/service_name`.

## 11.2. Warstwa dostępu - db.py (pomocnicze)

```
1 # Warstwa dostępu do bazy - python-oracledb (tryb thin, bez Instant Client).
2 # Operacje zapisujące idą przez pakiety PL/SQL (bez ORM).
3 # Odczyty to zwykle SELECT-y (często z widoków).
4
5 import oracledb
6 import config
7
8 _pool = None # pula połączeń tworzona przy pierwszym użyciu
9
10
11 def _get_pool():
12     global _pool
13     if _pool is None:
14         _pool = oracledb.create_pool(
15             user=config.DB_USER,
16             password=config.DB_PASSWORD,
17             dsn=config.DB_DSN,
18             min=1, max=4, increment=1,
19         )
20     return _pool
21
22
23 def polaczenie():
24     return _get_pool().acquire()
25
26
27 def wiersze_na_slovniki(cursor):
28     # zamienia wynik kursora na listę słowników {kolumna_małymi: wartosc}
29     kolumny = [c[0].lower() for c in cursor.description]
30     return [dict(zip(kolumny, w)) for w in cursor.fetchall()]
31
32
```

```

33 def _data(d):
34     return d.strftime("%Y-%m-%d") if d else ""
35
36
37 def czysty_blad(e):
38     # wyciaga czytelny komunikat z bledu Oracle (np. ORA-20003: brak na stanie)
39     try:
40         msg = e.args[0].message

```

- `_get_pool` – tworzy pule polaczen przy pierwszym uzyciu (leniwie),
- `_wiersze_na_slowniki` – zamienia wynik kursora na liste slownikow (klucz = nazwa kolumny),
- `czysty_blad` – wyciaga czytelny komunikat z bledu Oracle (tekst po “ORA-20003:”).

## Zapisy przez pakiety (bez ORM)

```

169 def zloz_zamowienie(id_klienta, adres, pozycje, metoda):
170     # pozycje: lista (id_produktu, ilosc). Tworzy zamowienie, dodaje pozycje, oplaca.
171     with polaczenie() as con:
172         cur = con.cursor()
173         v_zam = cur.callfunc("P_ZAMOWIENIA.zloz_zamowienie", oracledb.NUMBER,
174                             [id_klienta, adres])
175         for id_prod, ilosc in pozycje:
176             cur.callproc("P_ZAMOWIENIA.dodaj_pozycje", [int(v_zam), id_prod, ilosc])
177             cur.callproc("P_ZAMOWIENIA.dodaj_platnosc", [int(v_zam), metoda])
178         con.commit()
179         return int(v_zam)

```

`callfunc` wywołuje funkcje (np. `zloz_zamowienie` zwracająca ID), `callproc` – procedury (dodaj\_pozycje, dodaj\_platnosc). To realizacja wymagania “bez ORM, przez pakiety”. `con.commit()` zatwierdza transakcje.

## 11.3. Logowanie i bcrypt - app.py

```

171 def logowanie():
172     if request.method == "POST":
173         email = request.form.get("email", "")
174         haslo = request.form.get("haslo", "")
175         u = db.uzytkownik_po_emailu(email)
176         if u and u["aktywny"] == "T" and bcrypt.checkpw(haslo.encode(), u["haslo_hash"].encode()):
177             session["uzytkownik"] = {"id_klienta": u["id_klienta"], "email": u["email"],
178                                     "imie": u["imie"], "nazwisko": u["nazwisko"], "rola": u["rola"]}
179             flash(f"Witaj, {u['imie']}!", "success")
180             if u["rola"] in ("PRACOWNIK", "ADMIN"):
181                 return redirect(url_for("admin_panel"))
182             return redirect(url_for("index"))
183         flash("Bledny email lub haslo (albo konto nieaktywne).", "danger")
184     return render_template("login.html")
185
186
187 @app.route("/rejestracja", methods=["GET", "POST"])
188 def rejestracja():

```

- `db.uzytkownik_po_emailu` pobiera hash i role z bazy,
- `bcrypt.checkpw(haslo.encode(), u['haslo_hash'].encode())` porównuje wpisane hasło z hashem,
- sprawdzane jest też `aktywny == 'T'` – konto nieaktywne nie zaloguje się,

- po zalogowaniu rola decyduje o przekierowaniu (panel dla personelu, sklep dla klienta).

## 11.4. Szablony i wygląd

Szablony (katalog `templates/`) korzystają z dziedziczenia: `base.html` zawiera wspólny układ (pasek nawigacji, komunikaty, koszyk), a pozostałe strony go rozszerzają przez `{% extends %}`. Wygląd zapewnia Bootstrap 5 z CDN. Dostępne strony: katalog, produkt, koszyk, zamówienie, logowanie, rejestracja, moje zamówienia oraz panel (produkty, zamówienia, log).

## 12. Bezpieczeństwo

---

- **Hasła** – tylko hash bcrypt; bcrypt dodaje sol i jest celowo wolny, co utrudnia łamanie.
- **Role** – trzy role o różnym zakresie (klient/pracownik/admin), w hierarchii.
- **Widoki** – ukrywają dane (klient widzi tylko aktywne produkty przez v\_katalog).
- **Pakiety** – klient nie ma dostępu do tabel wprost, tylko przez pakiety – co kontroluje dozwolone operacje.
- **Audyt** – triggerzy zapisują każdą zmianę do LOG\_OPERACJI (kto, co, kiedy).
- **Ograniczenia** – CHECK/FK/UNIQUE pilnują poprawności danych na poziomie bazy.

## 13. Zgodnosc z wymaganiami

---

| Wymaganie                  | Realizacja                                                  |
|----------------------------|-------------------------------------------------------------|
| schemat bazy               | projekt.sql – 8 tabel, ograniczenia, indeksy                |
| aplikacja bazodanowa       | Flask + python-oracledb, operacje przez pakiety             |
| pakiety PL/SQL             | P_ZAMOWIENIA, P_KLIENCI                                     |
| triggery                   | kontrola stanu, magazyn po oplaceniu, audyt                 |
| rozne role z uprawnieniami | rola_klient / rola_pracownik / rola_admin + widoki          |
| hasla hashowane            | bcrypt (liczony w aplikacji)                                |
| bez ORM                    | zapisy przez callproc/callfunc, odczyty przez SELECT/widoki |
| prezentacja + wklad %      | Prezentacja_TechSklep.pptx (slajd z tabela wkladu)          |

## 14. Decyzje projektowe do obrony

---

Najwazniejsze swiadome decyzje (pelna lista w pliku AUDYT.md):

- **Odczyty przez widoki, zapisy przez pakiety** – interpretacja wymagania “bez ORM”.
- **Aplikacja laczy sie jako wlasciciel schematu** – role w bazie pokazuja model uprawnień, a rozroznienie w UI opiera sie na kolumnie rola.
- **Kolumna rola w KLIENCI** – jedna sciezka logowania dla klientow i personelu.
- **Bledy przez RAISE\_APPLICATION\_ERROR** – zamiast nazwanych wyjatkow; PRAGMA EXCEPTION\_INIT pominielo (nie bylo na zajeciach).
- **Zlozenie zamowienia od razu je oplaca** – uproszczenie; baza obsluguje tez osobna platnosc.

## 15. Mozliwe pytania na obronie

---

### **Dlaczego sekwencja + trigger, a nie kolumna IDENTITY?**

Bo taki wzorzec auto-inkrementacji byl uczony na wykladzie i dziala w kazdej wersji Oracle. Trigger BEFORE INSERT pobiera NEXTVAL, gdy ID nie zostalo podane.

### **Czym rozni sie funkcja od procedury?**

Funkcja zwraca wartosc (RETURN) i mozna jej uzyc w wyrazeniu; procedura wykonuje akcje. U nas zloz\_zamowienie i oblicz\_sume to funkcje, reszta to procedury.

### **Co to jest %TYPE i po co?**

To "typ jak kolumna". Parametr deklarowany jako kolumna.%TYPE zawsze pasuje do typu w bazie, wiec zmiana typu kolumny nie wymaga poprawek w kodzie.

### **Jak dziala kontrola magazynu?**

Dwuetapowo: przy dodawaniu pozycji trigger i pakiet sprawdzaja dostepnosc; przy zmianie statusu na OPLACONE trigger AFTER UPDATE zdejmuje towar, a przy anulowaniu – zwraca.

### **Co to mutating table i czy tu wystepuje?**

Blad, gdy trigger wierszowy odwołuje sie do tabeli, na ktorej wisi. Tu nie wystepuje – triggerry audytowe pisza do osobnej LOG\_OPERACJI, a trigger magazynowy modyfikuje PRODUKTY (inna tabela niz ZAMOWIENIA).

### **Roznica miedzy uprawnieniem systemowym a obiektowym?**

Systemowe pozwala na czynnosc w bazie (CREATE SESSION, CREATE TABLE); obiektowe dotyczy konkretnego obiektu (SELECT/EXECUTE ON ...). Role grupuja uprawnienia.

### **Dlaczego haslo jest hashowane w aplikacji, a nie w bazie?**

bcrypt to biblioteka aplikacyjna; baza dostaje gotowy hash. Haslo jawne nigdy nie trafia do bazy ani logow.

### **Po co kopiowac cene do pozycji zamowienia?**

Zeby pozniejsza zmiana ceny produktu nie zmienila wartosci juz zlozonych zamowien (zachowanie historii).

### **Czym jest idempotentnosc skryptow i jak ja zapewniono?**

To mozliwosc wielokrotnego uruchomienia bez bledow. Zapewniaja ja bloki DROP na poczatku, CREATE OR REPLACE oraz DELETE przed wstawieniem danych.

## 16. Instrukcja uruchomienia

---

### 16.1. Baza (SQL Developer)

Skrypty uruchamiamy jako właściciel schematu, w kolejności: projekt.sql, 02\_dane\_testowe.sql, 03\_pakiety.sql, 04\_triggery.sql, 05\_role.sql (wymaga CREATE ROLE / CREATE USER), 90\_testy.sql (opcjonalnie).

### 16.2. Aplikacja

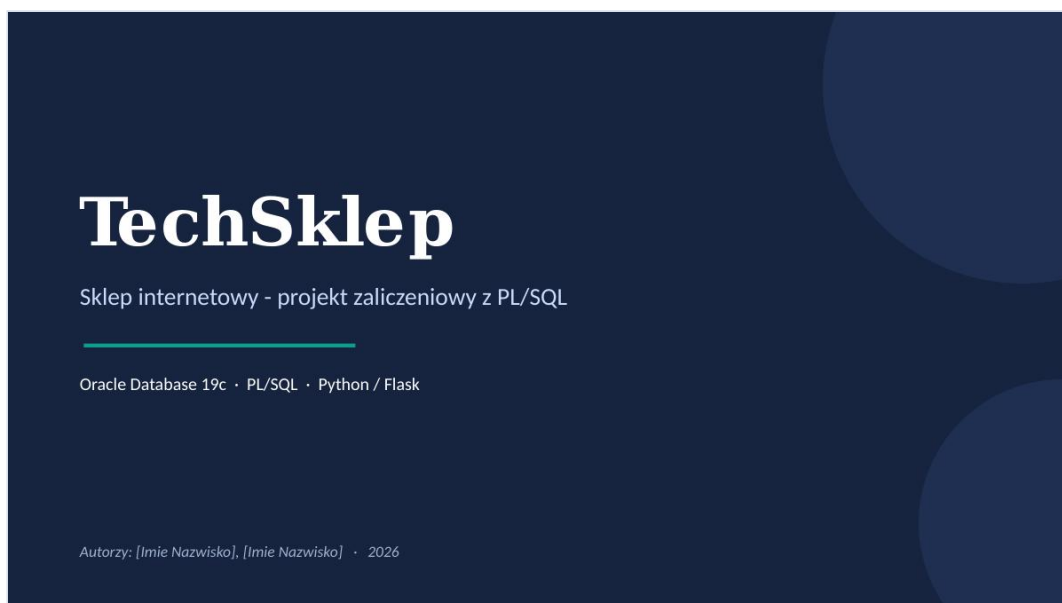
```
1 cd app
2 pip install -r requirements.txt
3 # ustaw dane polaczenia w config.py
4 python app.py
```

Aplikacja startuje na `http://127.0.0.1:5000`. Konta testowe: klient `anna.kowalska@example.com` / `haslo123`, pracownik `pracownik@techsklep.pl` / `praca123`, admin `admin@techsklep.pl` / `admin123`.



# 17. Prezentacja

Slajdy prezentacji obronnej (plik Prezentacja\_TechSklep.pptx):

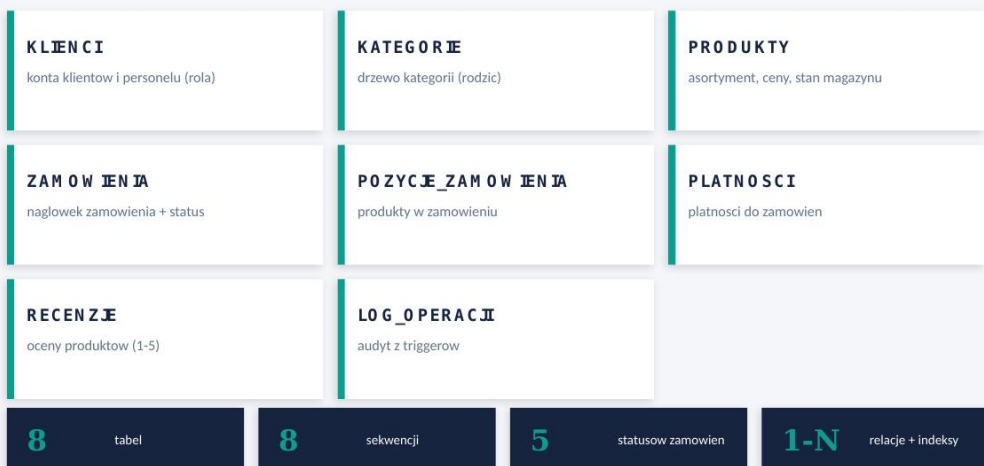


Slajd 1



Slajd 2

## 2 Schemat bazy danych



Slajd 3

## 3 Powiązania między tabelami

Główny przepływ danych w zamówieniu:



- KLIENCI 1 - N ZAMOWIENIA (klient składa wiele zamówień)
- ZAMOWIENIA 1 - N POZYCJE\_ZAMOWIENIA 1 - 1 PRODUKTY
- ZAMOWIENIA 1 - N PLATNOSCI
- PRODUKTY 1 - N RECENZJE N - 1 KLIENCI
- KATEGORIE 1 - N KATEGORIE (drzewo) oraz 1 - N PRODUKTY
- LOG\_OPERACJI - wypełniany automatycznie przez triggerzy audytowe

Slajd 4

## 4 Pakiety PL/SQL (logika aplikacji)

### P\_ZAMOWIENIA

|                                       |                                             |
|---------------------------------------|---------------------------------------------|
| <code>zloz_zamowienie()</code>        | tworzy zamówienie, zwraca ID                |
| <code>dodaj_pozycje()</code>          | dodaje produkt, kopiuje cene, sprawdza stan |
| <code>zmien_status()</code>           | kontrola dozwolonych przejść statusu        |
| <code>anuluj_zamowienie()</code>      | anuluje i zwraca towar na magazyn           |
| <code>oblicz_sume_zamowienia()</code> | funkcja licząca sumę                        |
| <code>dodaj_platnosc()</code>         | rejestruje płatność, ustawia OPLACONE       |

### P\_KLIENTI

`rejestruj()`  
zakłada konto klienta (hasło już zaszyfrowane  
bcryptem)

#### Własne wyjątki

-20001 klient nieaktywny  
-20002 produkt nieaktywny  
-20003 brak na stanie  
-20004 zły status  
-20005 brak zamówienia  
-20006 emalia zajęty

Slajd 5

## 5 Triggery - automatyzacja i audyt

1

### Kontrola stanu

Przy dodaniu pozycji sprawdza, czy  
starczy towaru. Brak -> błąd -20003.

2

### Zdejmowanie po opłaceniu

Gdy status zmienia się na OPLACONE,  
stan magazynowy maleje. Anulowanie  
zwraca towar.

3

### Audyt do LOG\_OPERACJI

Każdy INSERT/UPDATE/DELETE na  
kluczowych tabelach zapisuje wpis z  
USER i czasem.

Triggery audytowe zapisują do osobnej tabeli (LOG\_OPERACJI) - brak problemu mutating table.

Slajd 6

## 6 Role i uprawnienia

| Rola           | Uprawnienia                                                                           |
|----------------|---------------------------------------------------------------------------------------|
| rola_klient    | katalog (widok v_katalog), zakupy przez P_ZAMOWIENIA, rejestracja przez P_KLIENCI     |
| rola_pracownik | to co klient + podgląd wszystkich zamówień, zmiana statusów, podgląd produktów        |
| rola_admin     | to co pracownik + zarządzanie produktami/kategoriami, kontami i podgląd logu operacji |

Bezpieczeństwo realizowane przez ROLE + WIDOKI (zgodnie z materiałami z wykładu - bez VPD/kontekstów).

Slajd 7

## 7 Aplikacja kliencka (Flask)

- ✓ Katalog produktów z filtrem kategorii i wyszukiwarka
- ✓ Strona produktu z recenzjami i ocenami
- ✓ Koszyk i składanie zamówienia (przez pakiet)
- ✓ Logowanie / rejestracja - hasła bcrypt
- ✓ Panel pracownika/admina: produkty, zamówienia, log
- ✓ Rozdzielenie widoków wg roli użytkownika

### Kluczowa zasada

Aplikacja nie używa ORM.

Wszystkie operacje zapisujące (zamówienia, płatności, rejestracja, zmiana statusu) wykonywane są przez wywołania pakietów PL/SQL.

Slajd 8

## 8 Testy i scenariusze demonstracyjne

|   |                                  |                                         |
|---|----------------------------------|-----------------------------------------|
| 1 | Złożenie i opłacenie zamówienia  | -> stan magazynowy maleje automatycznie |
| 2 | Anulowanie opłaconego zamówienia | -> towar wraca na magazyn               |
| 3 | Proba zakupu ponad stan          | -> zgłaszany wyjątek -20003             |
| 4 | Niedozwolona zmiana statusu      | -> zgłaszany wyjątek -20004             |
| 5 | Operacje na danych               | -> wpisy pojawiają się w LOG_OPERACJI   |

Slajd 9

## 9 Podział pracy w grupie

| Członek grupy   | Zakres prac                               | Wkład |
|-----------------|-------------------------------------------|-------|
| [Imię Nazwisko] | schemat bazy, pakiety, triggerzy          | 50%   |
| [Imię Nazwisko] | role, aplikacja Flask, testy, prezentacja | 50%   |

(uzupełnić imionami i ewentualnie skorygować procenty)

Slajd 10

# Podsumowanie

- ✓ Znormalizowany schemat z nazwanymi constraintami i indeksami
- ✓ Pakiety PL/SQL z funkcjami, procedurami i własnymi wyjątkami
- ✓ Triggery: kontrola magazynu, automatyzacja, audyt
- ✓ Trzy role użytkowników z różnymi uprawnieniami + widoki
- ✓ Aplikacja webowa rozmawiająca z bazą tylko przez pakiety (bez ORM)

**Dziekujemy za uwagę**

*Slajd 11*